Chapter 10 — Behavioral Design Patterns (Part 1)

> **What you will learn.** What behavioral patterns are, and six of them in depth: **Chain of Responsibility, Command, Iterator, Mediator, Memento** and **Interpreter**. For each one you get a plain definition, the problem it fixes, the class diagram, Java 21 code that compiles and runs, real products and JDK classes that use it, and interview questions with answers. Chapter 11 covers the other five GoF behavioral patterns (Observer, State, Strategy, Template Method, Visitor) and some important patterns that are not in the GoF book.

How to run the code in this chapter

```
cd chapters/10-patterns-behavioral-1/code
javac --release 21 -d /tmp/lld_10 $(find . -name '*.java')
java -cp /tmp/lld_10 lld.ch10.chain.Main        # also: command, iterator, mediator, memento, interpreter
```

10.0 Behavioral patterns: the big picture

What it is

Creational patterns (Chapter 8) are about **how objects are made**. Structural patterns (Chapter 9) are about **how objects are put together**. **Behavioral patterns** are about **how objects share work and talk to each other**: who is responsible for what, and who calls whom.

Everyday analogy: a restaurant kitchen. The structural question is "which stations exist?" The behavioral question is "when an order comes in, who hears about it first, who passes it on, who decides the order of the dishes, and who can cancel it?"

Behavioral patterns help you to:

- let objects work together in a flexible, efficient way;
- cut tight coupling between classes, so each class is easier to change and test;
- turn tangled communication between many objects into something simple to follow;
- change behaviour **while the program runs** (at run time), not only when you compile it.

The eleven GoF behavioral patterns

The "Gang of Four" (GoF) book, *Design Patterns* (Gamma, Helm, Johnson, Vlissides, 1994), lists eleven behavioral patterns. This chapter covers the first six; Chapter 11 covers the rest.

Pattern	One-line intent	Chapter
Chain of Responsibility	Hand a request down a row of handler objects; each may deal with it, stop it, or forward it.	10
Command	Wrap a request in an object, which can then be stored, queued, logged or reversed.	10
Interpreter	Represent the grammar of a small language as classes, and evaluate sentences in it.	10
Iterator	Walk through a collection one element at a time without seeing how it is stored.	10
Mediator	Route communication between many objects through one coordinator.	10
Memento	Let an object save a snapshot of its own state and later return to it, while keeping that state private.	10
Observer	When one object changes, notify all the objects that depend on it.	11
State	Let an object change its behaviour when its internal state changes.	11
Strategy	Choose one of several interchangeable algorithms at run time.	11
Template Method	Fix the skeleton of an algorithm in a base class; let subclasses fill in some steps.	11
Visitor	Add a new operation to a set of classes without changing those classes.	11

Class scope vs object scope. GoF splits behavioral patterns into two kinds. *Class* behavioral patterns use **inheritance** to share behaviour: only **Template Method** and **Interpreter**. All the others are *object* behavioral patterns: they use **composition** (objects holding references to other objects) and can therefore be rewired at run time.

One idea runs through all of them: take a piece of behaviour that is hidden inside a big method (a chain of `if`s, a hard-coded call, a loop) and turn it into an **object** you can name, pass around, swap, store and test on its own.

10.1 Chain of Responsibility

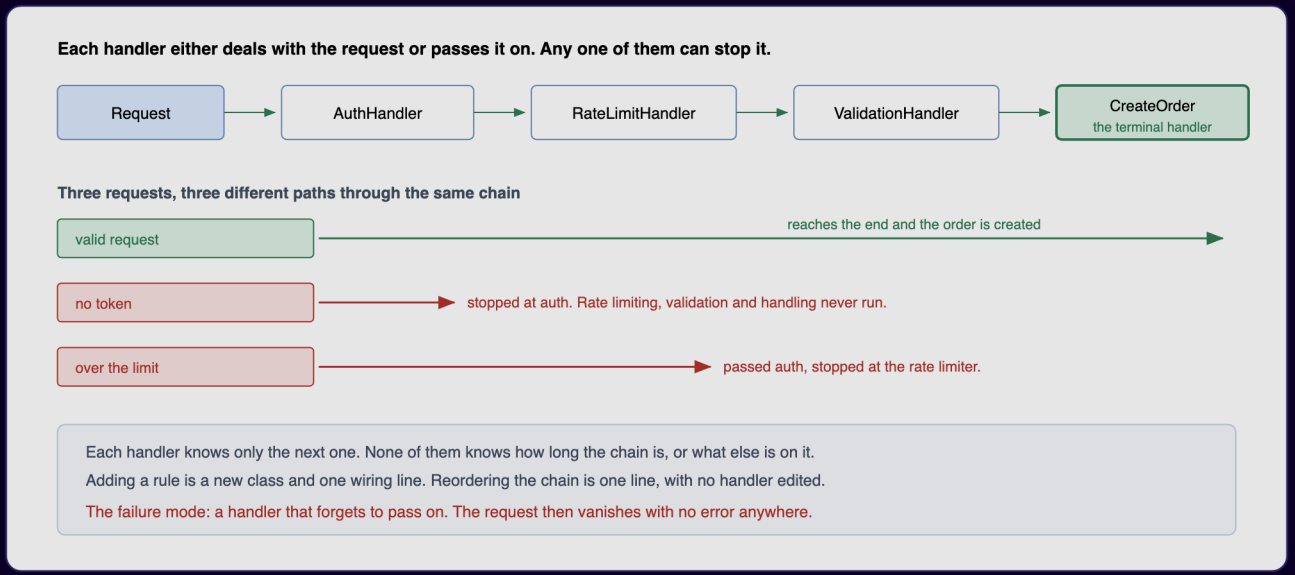


Figure – Three requests go into the same chain of handlers and each stops at a different place (course diagram)

What it is

Chain of Responsibility (CoR) sends a request through a row of **handler** objects. Every handler makes one of three choices: deal with the request, reject it (so nobody later sees it), or forward it to its successor. The sender talks only to the first handler and never learns which one finally took care of the request.

GoF intent, in plain words: *avoid tying the sender of a request to one particular receiver by giving several objects a chance to handle it; link those objects and forward the request until one handles it.* GoF participants: **Handler** (declares the handling method, optionally holds the successor link), **ConcreteHandler** (handles what it is responsible for, otherwise forwards), and **Client** (starts the request at a handler in the chain).

Everyday analogy: a customer-support phone line. Level-1 support tries first. If they cannot help, they pass you to level 2, then to an engineer. You only ever dialled one number.

In the figure above, one chain (auth → rate limit → validation → create order) receives three requests. A valid request reaches the end. A request with no token stops at auth. A request over the limit passes auth but stops at the rate limiter. Each handler knows only the next one.

The problem it solves

Before an order is created, a request must survive several gates: who is calling (authentication), are they calling too often (rate limit), and is the body well-formed (validation). Written directly, one method does all of them:

- The method grows every time a rule is added.
- The **order of the checks is hidden inside the method body**, yet it matters: authenticating before rate limiting is a different system from rate limiting before authenticating.
- Some routes need only two of the checks. Soon there are boolean flags, then a second copy of the method that someone forgets to update.

CoR makes **each check its own small object**, and links them into a line. No handler knows how long the line is or what else is on it, so **the line itself is configuration, not logic**: a new rule costs one new class plus one wiring line; changing the order touches only the wiring; each route can be given its own line, assembled when the application starts.

How it works

1. Define a **Handler** type with a method such as `handle(request)` and a reference to the **next** handler.
2. Write one **concrete handler** per check. Each one does its own check, then either **stops** (returns a result) or **passes on** (`next.handle(request)`).
3. Put a **terminal handler** at the end that does the real work (or reports "nobody handled this").
4. **Wire** the chain once, at start-up: `auth.setNext(rateLimit).setNext(validate).setNext(createOrder)`.
5. The client calls only the **first** handler. It never knows how long the chain is.

Structure (UML)

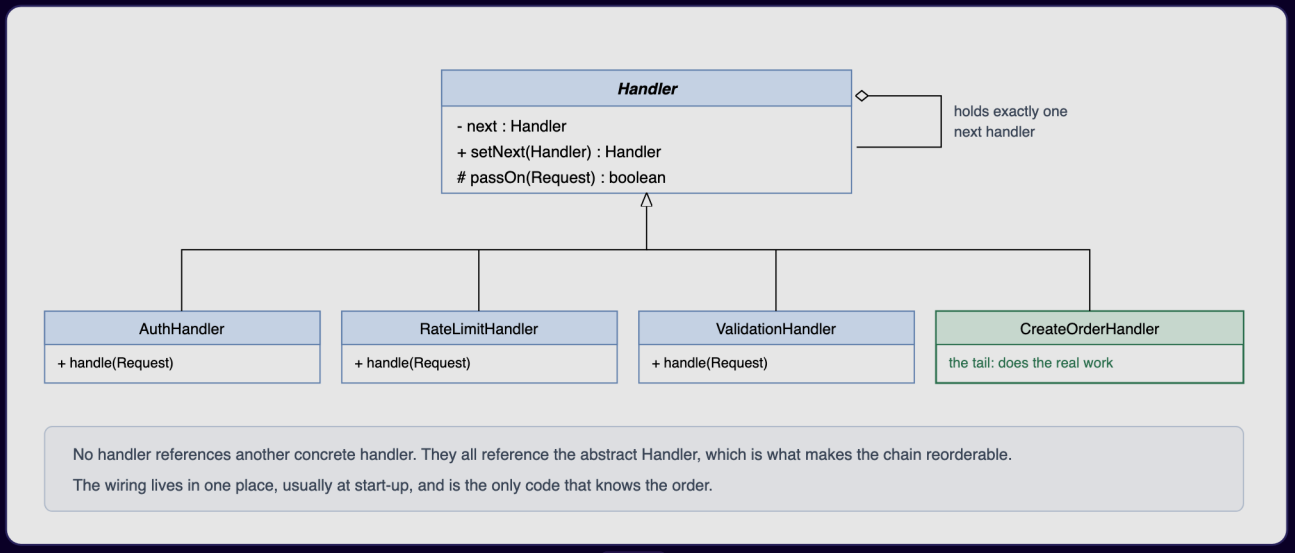


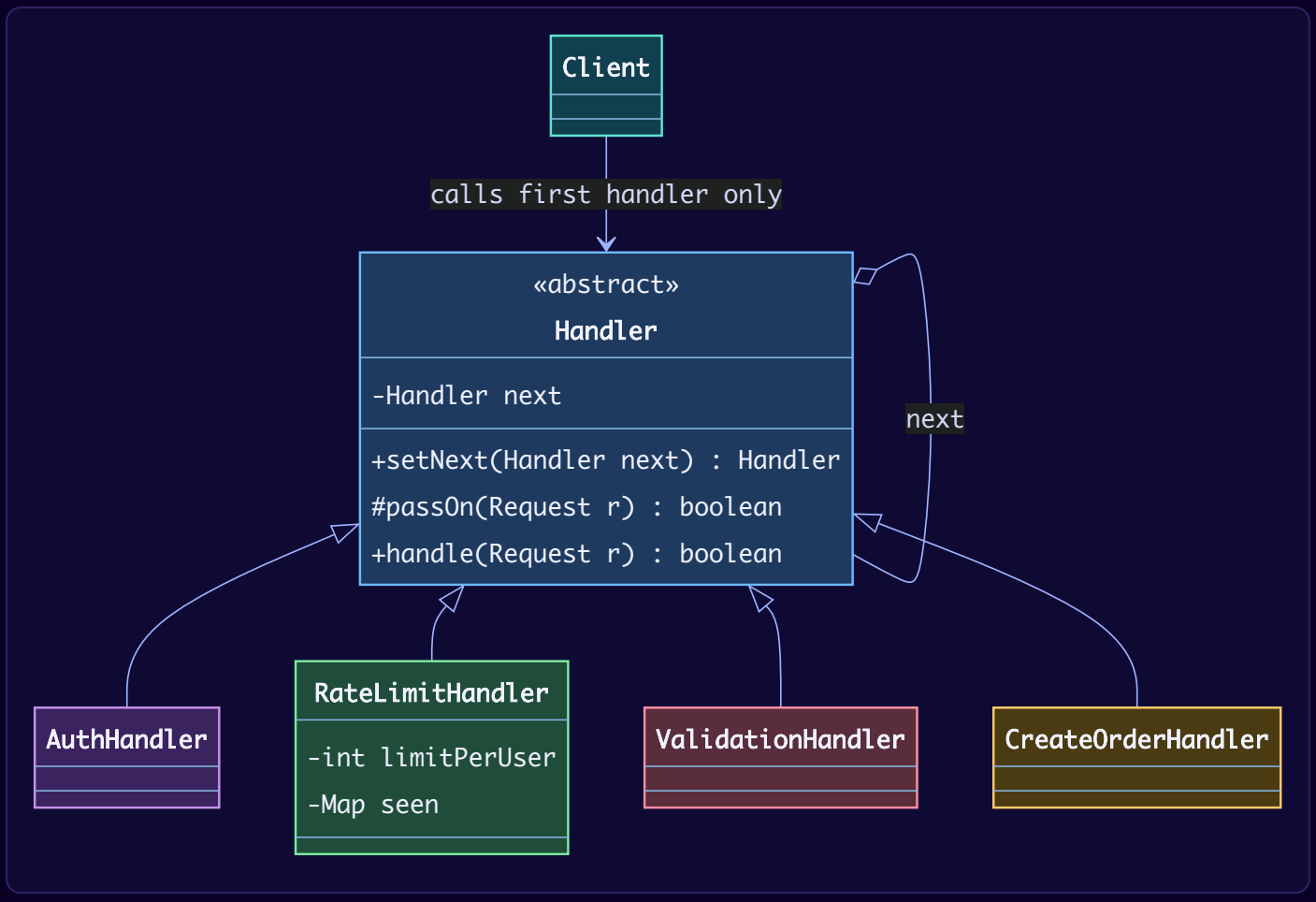
Figure — Class diagram: an abstract Handler holds `next`; concrete handlers extend it; CreateOrderHandler is the tail (course diagram)

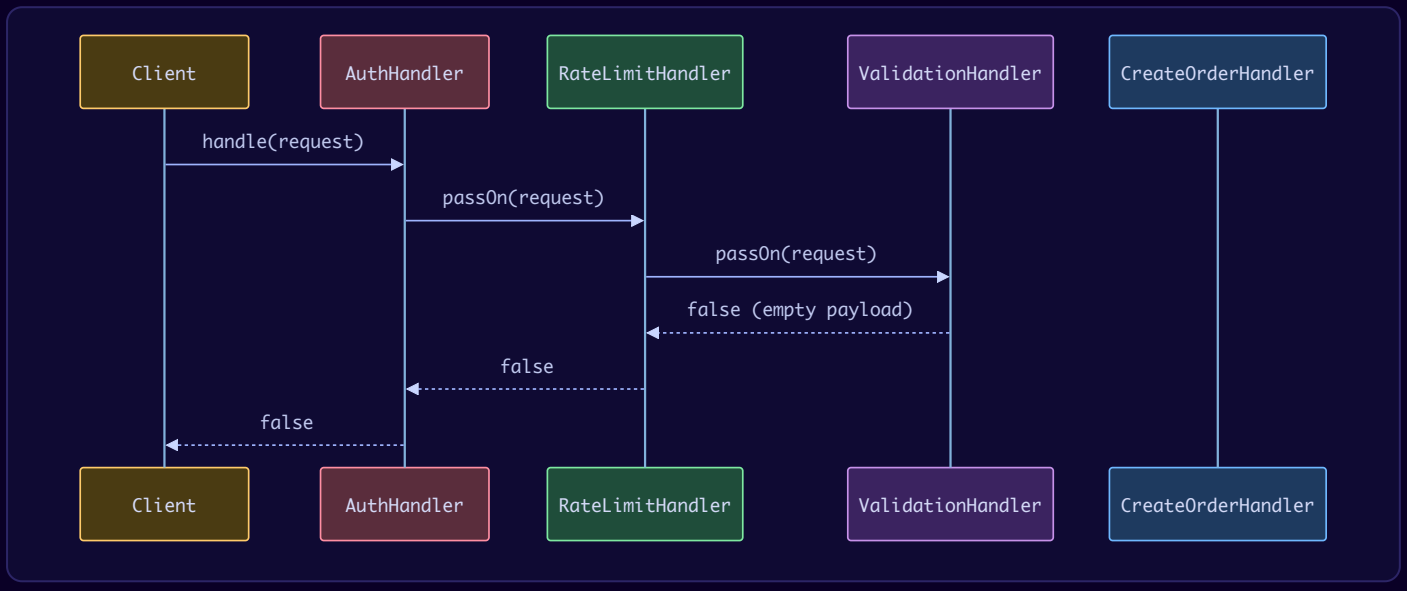
In the class diagram above, notice what is **missing**: no handler refers to another *concrete* handler. Every link is typed as the abstract `Handler`, and that is why the stages can be shuffled freely. `setNext` hands back its

argument, which lets the wiring be written as one fluent expression.

> **Our code vs the course pseudocode.** In the course version, `passOn` returns `true` ("success") when there is no next handler. Our `passOn` **throws** instead, because silently reporting success when the chain simply ran out is exactly the kind of invisible failure this pattern is prone to. Either choice is legal; just make it deliberate.

Our Java design:





Java example

The package `lld.ch10.chain` shows **four flavours** of the pattern:

1. **Classic linked handlers** (`Handler`, `AuthHandler`, `RateLimitHandler`, `ValidationHandler`, `CreateOrderHandler`). Each handler returns an *explicit* outcome: `false` = rejected, `passOn(...)` = continue. `passOn` **throws** if there is no next handler, so a chain without a terminal handler fails loudly instead of silently "succeeding".
2. **Approval chain** (`Approver`, `Expense`). A pure CoR: the *first* approver whose limit is high enough approves, and nobody after it sees the request. Note that there is only **one** class – the chain is just data (role + limit). Amounts and limits are `BigDecimal` compared with `compareTo` – never `double` for money.
3. **Servlet-style filters** (`Filter`, `FilterChain`, `Endpoint`, `HttpRequest`). This is how `jakarta.servlet.Filter` works: the "next" step is passed **in as an argument** (`chain.doFilter(...)`). A filter can act before *and after* the rest of the chain (see the timing filter), or stop the chain by not calling it (the auth filter).
4. **Logging levels** (`LogHandler`, `ConsoleLog`, `FileLog`, `PagerLog`, `Level`). A common *variant*: each handler acts if the message is severe enough and then **always** passes it on. Several handlers may act. Strictly, when nothing can stop the flow it is closer to a *pipeline*, but textbooks often show logging this way, and it is worth knowing the difference.

```java
// File: code/lld/ch10/chain/Handler.java
package lld.ch10.chain;

/** Classic Chain of Responsibility base: each handler knows only "the next one". */
public abstract class Handler {
    private Handler next;

    /** Links the next handler and returns it, so wiring reads as one chained expression. */
    public Handler setNext(Handler next) {
        this.next = next;
        return next;
    }

    /** Forwards the request. Falling off the end means the chain has no terminal handler: a wiring bug. */
    protected boolean passOn(Request request) {
        if (next == null) {
            throw new IllegalStateException(
                getClass().getSimpleName() + " tried to pass on, but the chain has no terminal handler");
        }
        return next.handle(request);
    }

    /** @return true if the request was fully handled, false if some handler rejected it. */
    public abstract boolean handle(Request request);
}
```

```java
// File: code/lld/ch10/chain/Request.java
package lld.ch10.chain;

/** The request that travels down the API pipeline. */
public record Request(String user, String token, String payload) {}
```

```java
// File: code/lld/ch10/chain/AuthHandler.java
package lld.ch10.chain;

/** Stage 1: reject requests with no token. */
final class AuthHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.token() == null || r.token().isBlank()) {
            System.out.println("  [auth] rejected: missing token");
            return false;                       // explicit outcome: the chain stops here
        }
        System.out.println("  [auth] ok for " + r.user());
        return passOn(r);                       // explicit outcome: continue
    }
}
```

```java
// File: code/lld/ch10/chain/RateLimitHandler.java
package lld.ch10.chain;

import java.util.HashMap;
import java.util.Map;

/** Stage 2: at most N requests per user (keyed by user because auth ran first). */
final class RateLimitHandler extends Handler {
    private final int limitPerUser;
    private final Map<String, Integer> seen = new HashMap<>();

    RateLimitHandler(int limitPerUser) { this.limitPerUser = limitPerUser; }

    @Override
    public boolean handle(Request r) {
        int count = seen.merge(r.user(), 1, Integer::sum);
        if (count > limitPerUser) {
            System.out.println("  [ratelimit] rejected: " + r.user() + " made " + count + " requests, limit "
+ limitPerUser);
            return false;
        }
        System.out.println("  [ratelimit] ok (" + count + "/" + limitPerUser + ")");
        return passOn(r);
    }
}
```

```java
// File: code/lld/ch10/chain/ValidationHandler.java
package lld.ch10.chain;

/** Stage 3: the payload must not be empty. */
final class ValidationHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        if (r.payload() == null || r.payload().isBlank()) {
            System.out.println("  [validate] rejected: empty payload");
            return false;
        }
        System.out.println("  [validate] ok");
        return passOn(r);
    }
}
```

```java
// File: code/lld/ch10/chain/CreateOrderHandler.java
package lld.ch10.chain;

/** Terminal handler: does the real work and never passes on. */
final class CreateOrderHandler extends Handler {
    @Override
    public boolean handle(Request r) {
        System.out.println("  [order] created order for " + r.user() + ": " + r.payload());
        return true;
    }
}
```

```java
// File: code/lld/ch10/chain/Expense.java
package lld.ch10.chain;

import java.math.BigDecimal;

/** An expense claim that needs someone with enough authority to approve it. Money is BigDecimal, never
double. */
record Expense(String id, BigDecimal amountUsd) {}
```

```java
// File: code/lld/ch10/chain/Approver.java
package lld.ch10.chain;

import java.math.BigDecimal;

/**
 * Approval chain. Pure Chain of Responsibility: the FIRST handler able to deal
 * with the request handles it, and nobody after it sees the request.
 * Note there is only one class: the chain is data (name + limit), not code.
 */
final class Approver {
    private final String role;
    private final BigDecimal limitUsd;
    private Approver next;

    Approver(String role, BigDecimal limitUsd) {
        this.role = role;
        this.limitUsd = limitUsd;
    }

    Approver then(Approver next) {
        this.next = next;
        return next;
    }

    String approve(Expense e) {
        if (e.amountUsd().compareTo(limitUsd) <= 0) {   // compareTo, not equals: 300 and 300.00 are the same
amount
            return role + " approved " + e.id() + " ($" + e.amountUsd() + ")";
        }
        if (next == null) {   // explicit terminal outcome instead of a silent "nothing happened"
            return "REJECTED " + e.id() + " ($" + e.amountUsd() + "): above every approver's limit";
        }
        System.out.println("    " + role + " (limit $" + limitUsd + ") escalates " + e.id());
        return next.approve(e);
    }
}
```

```java
// File: code/lld/ch10/chain/HttpRequest.java
package lld.ch10.chain;

import java.util.Map;

/** Minimal HTTP request for the servlet-style demo. */
record HttpRequest(String path, Map<String, String> headers) {}
```

```java
// File: code/lld/ch10/chain/Filter.java
package lld.ch10.chain;

/** Same shape as jakarta.servlet.Filter: "next" is an explicit argument, so forgetting it is visible. */
@FunctionalInterface
interface Filter {
    void doFilter(HttpRequest request, StringBuilder response, FilterChain chain);
}
```

```java
// File: code/lld/ch10/chain/Endpoint.java
package lld.ch10.chain;

/** The endpoint at the end of the chain (like a Servlet). */
@FunctionalInterface
interface Endpoint {
    void service(HttpRequest request, StringBuilder response);
}
```

```java
// File: code/lld/ch10/chain/FilterChain.java
package lld.ch10.chain;

import java.util.List;

/** Walks the filter list. One FilterChain object per request, because it holds a position. */
final class FilterChain {
    private final List<Filter> filters;
    private final Endpoint endpoint;
    private int position = 0;

    FilterChain(List<Filter> filters, Endpoint endpoint) {
        this.filters = filters;
        this.endpoint = endpoint;
    }

    void doFilter(HttpRequest request, StringBuilder response) {
        if (position < filters.size()) {
            filters.get(position++).doFilter(request, response, this);
        } else {
            endpoint.service(request, response);   // the terminal handler
        }
    }
}
```

```java
// File: code/lld/ch10/chain/Level.java
package lld.ch10.chain;

/** Severity levels, ordered from least to most severe. */
enum Level { DEBUG, INFO, ERROR }
```

```java
// File: code/lld/ch10/chain/LogHandler.java
package lld.ch10.chain;

/**
 * Logging-level chain. A VARIANT of the pattern: each handler acts if the record
 * is severe enough, and then ALWAYS passes it on, so several handlers may act.
 * (If nobody can stop the flow, it is really a pipeline; the classic form stops.)
 */
abstract class LogHandler {
    private final Level threshold;
    private LogHandler next;

    LogHandler(Level threshold) { this.threshold = threshold; }

    LogHandler setNext(LogHandler next) {
        this.next = next;
        return next;
    }

    final void log(Level level, String message) {
        if (level.compareTo(threshold) >= 0) {
            write(level, message);
        }
        if (next != null) {
            next.log(level, message);
        }
    }

    protected abstract void write(Level level, String message);
}
```

```java
// File: code/lld/ch10/chain/ConsoleLog.java
package lld.ch10.chain;

final class ConsoleLog extends LogHandler {
    ConsoleLog() { super(Level.DEBUG); }
    @Override protected void write(Level l, String m) { System.out.println("    console <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/FileLog.java
package lld.ch10.chain;

final class FileLog extends LogHandler {
    FileLog() { super(Level.INFO); }
    @Override protected void write(Level l, String m) { System.out.println("    app.log <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/PagerLog.java
package lld.ch10.chain;

final class PagerLog extends LogHandler {
    PagerLog() { super(Level.ERROR); }
    @Override protected void write(Level l, String m) { System.out.println("    PAGER   <- " + l + " " + m); }
}
```

```java
// File: code/lld/ch10/chain/Main.java
package lld.ch10.chain;

import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. API request pipeline (classic linked handlers) ===");
        // Order is a design decision: auth BEFORE rate limiting so the limiter can key on the user,
        // and so anonymous traffic cannot burn a real user's quota.
        Handler pipeline = new AuthHandler();
        pipeline.setNext(new RateLimitHandler(2))
                .setNext(new ValidationHandler())
                .setNext(new CreateOrderHandler());

        List<Request> requests = List.of(
            new Request("alice", "t-123", "2 x widget"),
            new Request("bob", "", "1 x gadget"),
            new Request("alice", "t-123", " "),
            new Request("alice", "t-123", "1 x gadget"));
        for (Request r : requests) {
            System.out.println("request from " + r.user() + ":");
            System.out.println("  => handled = " + pipeline.handle(r));
        }

        System.out.println("\n=== 2. Expense approval chain (first capable handler wins) ===");
        Approver chain = new Approver("TeamLead", new BigDecimal("1000"));
        chain.then(new Approver("Manager", new BigDecimal("10000")))
             .then(new Approver("Director", new BigDecimal("50000")));
        for (Expense e : List.of(new Expense("EXP-1", new BigDecimal("300")),
                                 new Expense("EXP-2", new BigDecimal("7500")),
                                 new Expense("EXP-3", new BigDecimal("120000")))) {
            System.out.println("  " + chain.approve(e));
        }

        System.out.println("\n=== 3. Servlet-style filters (next is an explicit argument) ===");
        Filter timing = (req, res, next) -> {
            System.out.println("  [timing] start " + req.path());
            next.doFilter(req, res);                        // filters can act before AND after
            System.out.println("  [timing] end   " + req.path() + " -> " + res);
        };
        Filter auth = (req, res, next) -> {
            if (!req.headers().containsKey("Authorization")) {
                res.append("401 Unauthorized");             // does not call next: chain stops
                return;
            }
            next.doFilter(req, res);
        };
        Endpoint ordersEndpoint = (req, res) -> res.append("200 OK orders list");

        for (HttpRequest req : List.of(
                new HttpRequest("/orders", Map.of("Authorization", "Bearer abc")),
                new HttpRequest("/orders", Map.of()))) {
            new FilterChain(List.of(timing, auth), ordersEndpoint).doFilter(req, new StringBuilder());
        }

        System.out.println("\n=== 4. Logging levels (variant: every handler may act) ===");
        LogHandler logger = new ConsoleLog();
        logger.setNext(new FileLog()).setNext(new PagerLog());
        System.out.println("  log DEBUG:");
        logger.log(Level.DEBUG, "cache miss for key=42");
        System.out.println("  log ERROR:");
        logger.log(Level.ERROR, "payment service down");
```

```
    }
}
```

Output:

```
=== 1. API request pipeline (classic linked handlers) ===
request from alice:
  [auth] ok for alice
  [ratelimit] ok (1/2)
  [validate] ok
  [order] created order for alice: 2 x widget
  => handled = true
request from bob:
  [auth] rejected: missing token
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] ok (2/2)
  [validate] rejected: empty payload
  => handled = false
request from alice:
  [auth] ok for alice
  [ratelimit] rejected: alice made 3 requests, limit 2
  => handled = false

=== 2. Expense approval chain (first capable handler wins) ===
  TeamLead approved EXP-1 ($300)
    TeamLead (limit $1000) escalates EXP-2
  Manager approved EXP-2 ($7500)
    TeamLead (limit $1000) escalates EXP-3
    Manager (limit $10000) escalates EXP-3
  REJECTED EXP-3 ($120000): above every approver's limit

=== 3. Servlet-style filters (next is an explicit argument) ===
  [timing] start /orders
  [timing] end   /orders -> 200 OK orders list
  [timing] start /orders
  [timing] end   /orders -> 401 Unauthorized

=== 4. Logging levels (variant: every handler may act) ===
  log DEBUG:
    console <- DEBUG cache miss for key=42
  log ERROR:
    console <- ERROR payment service down
    app.log <- ERROR payment service down
    PAGER   <- ERROR payment service down
```

Walk through the output:

- Request 1 travels the full chain. Request 2 stops at auth – the rate limiter, validator and order code never run. Request 3 passes auth and the limiter (2 of 2) and stops at validation. Request 4 is the third request from alice, so the limiter stops it.
- `EXP-2` is escalated once and approved by the Manager. `EXP-3` is above every limit, so the chain gives an explicit "REJECTED" outcome rather than disappearing.
- The timing filter prints both before and after the rest of the chain. For the unauthenticated request, the auth filter wrote `401` and did not call `next`, so the endpoint never ran.
- A `DEBUG` message only reaches the console; an `ERROR` goes to all three.

Trade-offs

Pros	Cons
A new stage costs one class and one wiring line; no working class is edited (Open/Closed).	To see the route a request takes you must read the wiring plus several files.
Every stage can be unit-tested on its own against a stub successor.	When a request is refused, you may have to step through several handlers to learn which one refused it.
Each route or environment can get its own chain, assembled at start-up.	No compiler or framework verifies that the chain is complete or correctly ordered.
The sender does not know which object will end up handling its request.	A handler that neither handles nor forwards makes the request vanish without any error.
	Each extra stage adds a call; rarely matters, but can on very hot paths with long chains.

The order of the stages is a design decision that no tool verifies. Auth before rate limiting means the limiter can count per *user*, and requests without valid credentials are turned away before they can use up a real user's quota. Rate limiting before auth means the limiter can only key on something like the IP address – but it shields the (possibly expensive) auth step from floods. Both are reasonable; they behave differently. **Put a comment with the reason beside the wiring** (see `Main`): the order is written down nowhere else.

When not to use it

- **Every request always goes through every stage in a fixed order.** Four calls in one method are clearer and have no wiring to get wrong.
- **Handlers need each other's results.** Then the stages are not independent: what you really have is a *pipeline* in which each step reshapes a value for the next. Call the steps one after another and pass an accumulating context object between them instead.
- **No terminal handler exists.** If the end of the chain can be reached with nothing done, failures become silent. Add an explicit terminal handler first.
- **The chain is long and on a very hot path**, where every extra call is measurable.

Common mistakes

1. **The swallowing bug.** A handler does its check and then just `return`s instead of calling `passOn`. No exception, no log; orders "sometimes don't appear". Fixes: give every path an explicit outcome (handled / rejected with reason / passed on); prefer the style where `next` is an explicit parameter (servlet filters, Express, Go `http.Handler`) so forgetting it is visible; unit-test each handler against a stub successor that remembers whether it got the request.
2. **No terminal handler.** Reaching the end should be an error or a clear "not handled" result – never a shrug.
3. **Wrong order, undocumented.** See the trade-offs above.
4. **Handlers that know about each other** (`new RateLimitHandler()` inside `AuthHandler`). The chain is then fixed in code again.
5. **Sharing a stateful chain object between requests.** The servlet-style `FilterChain` keeps a position, so it must be **one per request** (our `Main` creates a new one for each).

Real-world examples

- **Servlet filters**: `jakarta.servlet.Filter.doFilter(request, response, chain)` – calling `chain.doFilter(...)` *is* "pass on". **Spring Security** is a chain of such filters (`SecurityFilterChain`).
- **Spring MVC** `HandlerInterceptor`: `preHandle` returns `false` to stop the request.
- **HTTP middleware** in Express (`next()`), Django, Ruby's Rack, ASP.NET Core, Go (`func(http.Handler) http.Handler`, the usual way to structure a Go web service). **OkHttp** interceptors call `chain.proceed(request)`. **Netty**'s `ChannelPipeline` passes events from handler to handler.
- **In languages with first-class functions** the chain is usually a list of `(request, next) => ...` functions rather than classes; JavaScript frameworks often build it by folding the list with `Array.reduce`, and in Python several decorators stacked on one view function form a chain written as syntax. Passing `next` explicitly removes the "forgot to pass on" bug by design.

- `java.util.logging`: a `LogRecord` is published to the logger's handlers and then, by default, to its parent logger's handlers; `setUseParentHandlers(false)` stops that. This is why setting a level or handler on a parent logger affects its children.
- **Exception handling**: each stack frame either catches the exception or lets it travel up — the language's own chain.
- **DOM event bubbling**: a click event climbs from the clicked element through its parent elements; any listener can call `stopPropagation()` to end the climb.
- **Approval workflows** (expenses, leave requests, loan approvals) where authority is based on amount or role, and **customer-support escalation** tiers.

In the interview

> **30-second answer:** "Chain of Responsibility splits a series of checks into separate handler objects, each linked only to its successor. The request moves down the line until some handler completes it or refuses it. Because only the wiring knows the order, adding, removing or reordering a rule is cheap. Servlet filters and HTTP middleware are the everyday examples. The classic bug is a handler that forgets to pass on and swallows the request, so I give every path an explicit outcome and put a terminal handler at the end."

Follow-ups:

- *How is it different from Decorator?* Both link objects that share one interface. With a **Decorator**, all wrappers run and the call is always delivered to the inner object; the point is to *add* behaviour. With **CoR**, a handler may **end** the request, and handlers further down never see it.
- *How is it different from Observer?* Observer tells **every** subscriber, and no subscriber can block the rest. CoR ends at the **first** handler that accepts the job.
- *How is it different from a pipeline?* A pipeline pushes a value through **all** of its steps and produces a transformed result. A chain *routes* a request to whichever handler takes it. Rule of thumb: if no stage can stop the flow, you have a pipeline.
- *Combine it with Command?* Yes — the request moving through the handlers is frequently itself a Command object (e.g. a job checked by validation and authorisation handlers before a worker runs it).
- *What if no handler can handle the request?* Put an explicit terminal handler (e.g. `NoHandlerFound`) that returns an error or logs, so failures are never silent.
- *How do you test it?* Test each handler with a fake next handler that records whether it was called and with what; then one integration test for the wiring order.
- *Where should rate limiting go, before or after auth?* Explain the trade-off above and say you would document the reason next to the wiring.

Self-check

1. **Q:** What does each handler hold a reference to? **A:** Only the next handler (through the abstract `Handler` type).
2. **Q:** Name the most common CoR bug. **A:** A handler that neither handles nor passes on — the request silently disappears.
3. **Q:** Why do modern frameworks pass `next` as a parameter? **A:** Because a missing `next(...)` call is then easy to spot in the handler's own body, and because the handler can do work both before and after the rest of the chain runs.
4. **Q:** Is a logging chain where every handler always passes on a "pure" CoR? **A:** Not strictly; since no handler can stop the flow it behaves like a pipeline. It is a common variant.
5. **Q:** When should you *not* use CoR? **A:** When every stage always runs in a fixed order, when stages depend on each other's results, or when there is no terminal handler.

TL;DR

A request walks a line of handlers; each handles, rejects, or passes on. The chain is data, so it is easy to change — but **order matters**, **always have a terminal handler**, and **never swallow a request**.

10.2 Command

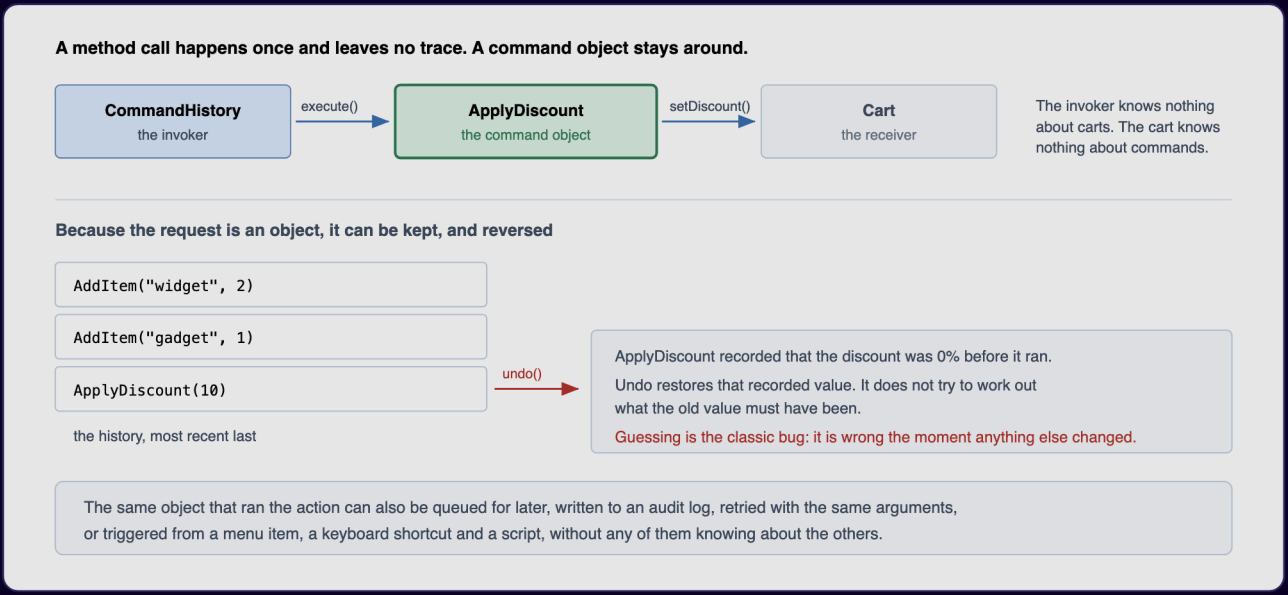


Figure — The invoker (CommandHistory) runs command objects against the receiver (Cart) and keeps them so they can be undone (course diagram). The figure assumes the discount was 0% before; our code deliberately starts at 5% to prove undo restores the recorded value.

What it is

Command packages a request ("add 2 widgets to the cart") as an **object** containing everything required to perform it later. Once the request is a value you can hold, you can **keep it in a list, put it on a queue, write it to an audit log, run it again, hand it to a worker thread** — and, when it also knows its own inverse, **reverse it**.

GoF intent, in plain words: *wrap a request in an object, so clients can be given different requests, requests can be queued or logged, and operations can be undone.*

Everyday analogy: a restaurant order slip. The waiter (invoker) does not cook; they write the order on a slip (command) and put it on the queue. The cook (receiver) acts on it later. The slip can be queued, prioritised, or cancelled.

The problem it solves

A plain method call such as `cart.setDiscount(10)` runs immediately and is forgotten the moment it returns. That is not enough once you need any of these:

- **Undo** — "reverse my last two changes".
- **A queue** — run the work later, on a worker thread, not in the request thread.
- **A log of intent** — an auditor asks which operations were *requested*, not just what state the data ended up in.
- **Retry** — run a failed action again with the same arguments.
- **One action, many triggers** — a menu item, a hot-key and an automation script should all fire the same "apply discount".

Command gives the request a body that outlives the call.

How it works

Interviewers usually want **three roles** named — **invoker**, **command**, **receiver** — plus the **client** that creates commands. GoF lists five participants in total: **Command** (the interface), **ConcreteCommand** (binds a receiver to an action), **Client**, **Invoker** and **Receiver**:

Role	Job	In our code
Command	Interface with `execute()` (and optionally `undo()`).	`Command`, `UndoableCommand`
Concrete command	Stores its receiver and arguments; can perform (and reverse) exactly one operation.	`AddItemCommand`, `ApplyDiscountCommand`, `MacroCommand`
Receiver	The object whose state is really modified. Has no idea commands exist.	`Cart`
Invoker	Triggers commands and keeps a record of them. Has no idea carts exist.	`CommandHistory`, `ExecutorService`
Client	Builds a concrete command and gives it its receiver.	`Main`

Steps:

1. The client creates a concrete command, passing the **receiver** and the **arguments**.
2. It hands the command to the **invoker**.
3. The invoker calls `execute()` and, if the command is undoable, pushes it on an **undo stack**.
4. **Undo** = pop the last command and call `undo()`; push it on a **redo stack**.
5. **Redo** = pop from redo, `execute()` again, push back on undo. **Running a fresh command empties the redo stack**: the branch of history you might have re-applied has been replaced.

Structure (UML)

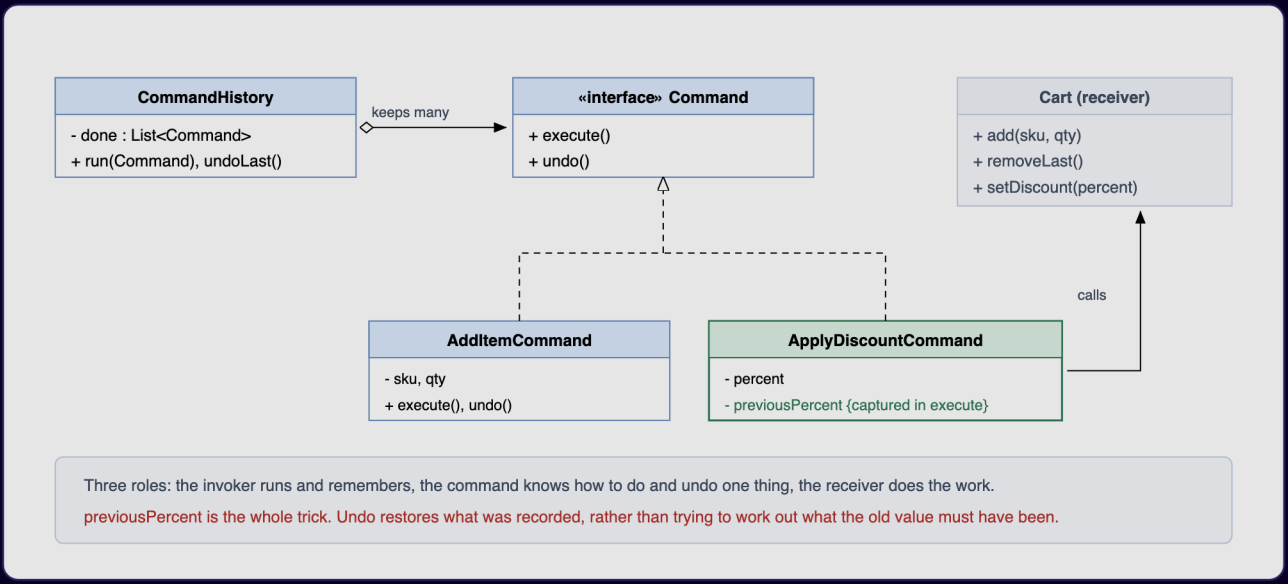
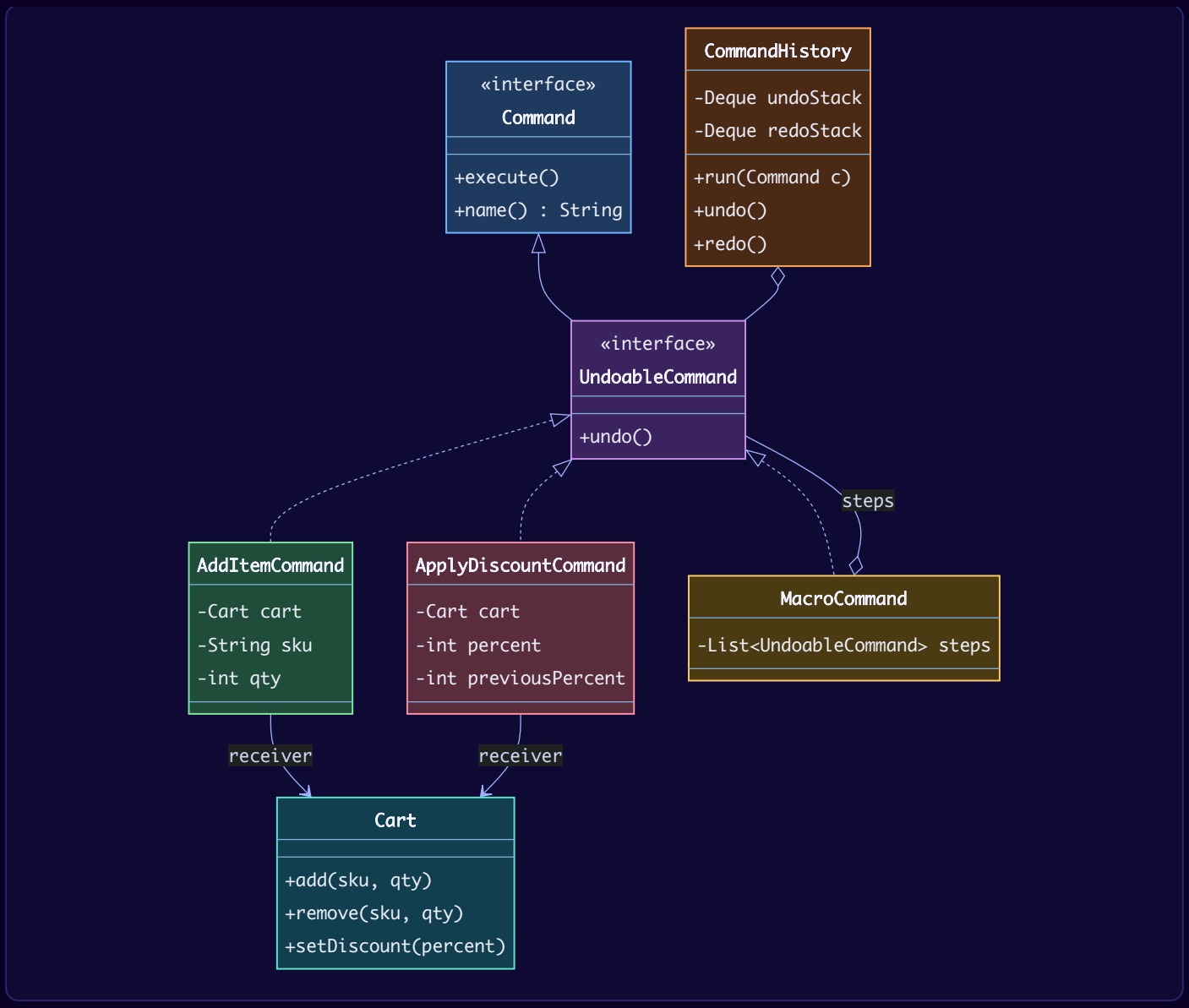
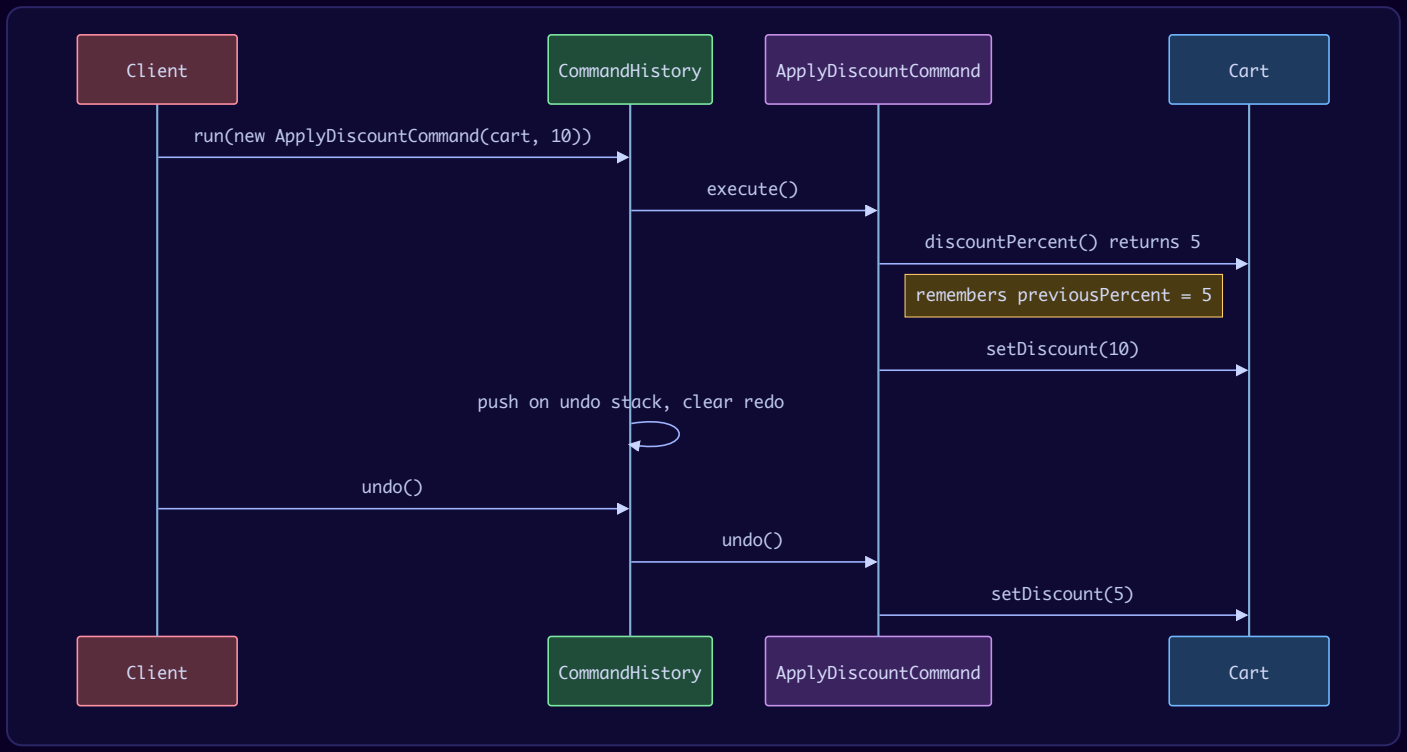


Figure – Class diagram: Command interface with execute/undo, two concrete commands, Cart as receiver, CommandHistory as invoker (course diagram; its `undoLast` / `removeLast` are named `undo` / `remove` in our code)

In the diagram above, each command keeps its own reference to the `Cart`. It has to: when it finally runs, the code that created it may have finished long ago, so the target object must travel inside the command along with the arguments. Also note `ApplyDiscountCommand.previousPercent` – the single field that makes its undo correct. And note that `CommandHistory` never mentions `Cart`; hand it any other commands and it still works, which is why one history can serve a whole application.

Our Java design:


«interface»
Command
+execute()
+name() : String
CommandHistory
-Deque undoStack
-Deque redoStack
+run(Command c)
+undo()
+redo()
«interface»
UndoableCommand
+undo()
steps
AddItemCommand
-Cart cart
-String sku
-int qty
ApplyDiscountCommand
-Cart cart
-int percent
-int previousPercent
MacroCommand
-List<UndoableCommand> steps
receiver
receiver
Cart
+add(sku, qty)
+remove(sku, qty)
+setDiscount(percent)


Client
CommandHistory
ApplyDiscountCommand
Cart
run(new ApplyDiscountCommand(cart, 10))
execute()
discountPercent() returns 5
remembers previousPercent = 5
setDiscount(10)
push on undo stack, clear redo
undo()
undo()
setDiscount(5)
Client
CommandHistory
ApplyDiscountCommand
Cart

Java example

Key design points in `lld.ch10.command`:

- `Command` has **one abstract method**, so any lambda or `Runnable`-like block can be a command. Only commands that can *honestly* be reversed implement `UndoableCommand`. `CommandHistory` keeps irreversible commands out of the undo stack.
- `ApplyDiscountCommand` **captures the old value in** `execute()` and restores exactly that in `undo()`. The cart deliberately starts at 5%, to show that "undo = set to 0" would be wrong.
- `MacroCommand` is a command made of commands (a **Composite** of commands): it executes steps in order and undoes them in **reverse** order.
- Section 4 of `Main` shows the JDK's own version: `Runnable` is the command interface and an `ExecutorService` is the invoker that runs commands on a worker thread.

```java
// File: code/lld/ch10/command/Command.java
package lld.ch10.command;

/** A request turned into an object. One abstract method, so a lambda is a Command. */
@FunctionalInterface
public interface Command {
    void execute();

    default String name() { return getClass().getSimpleName(); }
}
```

```java
// File: code/lld/ch10/command/UndoableCommand.java
package lld.ch10.command;

/** Only commands that can honestly be reversed implement this. */
public interface UndoableCommand extends Command {
    void undo();
}
```

```java
// File: code/lld/ch10/command/Cart.java
package lld.ch10.command;

import java.util.ArrayList;
import java.util.List;

/** The RECEIVER: knows how to change itself and nothing about commands. */
public final class Cart {
    public record Line(String sku, int qty) {}

    private final List<Line> lines = new ArrayList<>();
    private int discountPercent = 0;

    public void add(String sku, int qty) { lines.add(new Line(sku, qty)); }

    /** Removes the most recent line matching sku and qty. */
    public void remove(String sku, int qty) {
        for (int i = lines.size() - 1; i >= 0; i--) {
            if (lines.get(i).equals(new Line(sku, qty))) {
                lines.remove(i);
                return;
            }
        }
    }

    public int discountPercent() { return discountPercent; }
    public void setDiscount(int percent) { this.discountPercent = percent; }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder("Cart[");
        for (int i = 0; i < lines.size(); i++) {
            if (i > 0) sb.append(", ");
            sb.append(lines.get(i).sku()).append(" x").append(lines.get(i).qty());
        }
        return sb.append("] discount=").append(discountPercent).append('%').toString();
    }
}
```

```java
// File: code/lld/ch10/command/AddItemCommand.java
package lld.ch10.command;

/** Adds a line; undo removes exactly that line. */
final class AddItemCommand implements UndoableCommand {
    private final Cart cart;      // the command carries its receiver...
    private final String sku;     // ...and all the arguments it needs to run later
    private final int qty;

    AddItemCommand(Cart cart, String sku, int qty) {
        this.cart = cart;
        this.sku = sku;
        this.qty = qty;
    }

    @Override public void execute() { cart.add(sku, qty); }
    @Override public void undo()    { cart.remove(sku, qty); }
    @Override public String name()  { return "AddItem(" + sku + ", " + qty + ")"; }
}
```

```java
// File: code/lld/ch10/command/ApplyDiscountCommand.java
package lld.ch10.command;

/** Sets a discount; captures the previous value during execute so undo RESTORES, never guesses. */
final class ApplyDiscountCommand implements UndoableCommand {
    private final Cart cart;
    private final int percent;
    private int previousPercent;          // filled in by execute()

    ApplyDiscountCommand(Cart cart, int percent) {
        this.cart = cart;
        this.percent = percent;
    }

    @Override
    public void execute() {
        previousPercent = cart.discountPercent();   // capture BEFORE changing
        cart.setDiscount(percent);
    }

    @Override public void undo()   { cart.setDiscount(previousPercent); }
    @Override public String name() { return "ApplyDiscount(" + percent + "%)"; }
}
```

```java
// File: code/lld/ch10/command/MacroCommand.java
package lld.ch10.command;

import java.util.List;

/** A command made of commands. Runs in order; undoes in REVERSE order. */
final class MacroCommand implements UndoableCommand {
    private final String label;
    private final List<UndoableCommand> steps;

    MacroCommand(String label, List<UndoableCommand> steps) {
        this.label = label;
        this.steps = List.copyOf(steps);
    }

    @Override public void execute() { steps.forEach(Command::execute); }

    @Override
    public void undo() {
        for (int i = steps.size() - 1; i >= 0; i--) steps.get(i).undo();
    }

    @Override public String name() { return "Macro:" + label; }
}
```

```java
// File: code/lld/ch10/command/CommandHistory.java
package lld.ch10.command;

import java.util.ArrayDeque;
import java.util.Deque;

/** The INVOKER: runs commands and remembers them. Knows nothing about carts. */
public final class CommandHistory {
    private final Deque<UndoableCommand> undoStack = new ArrayDeque<>();
    private final Deque<UndoableCommand> redoStack = new ArrayDeque<>();

    public void run(Command command) {
        command.execute();
        if (command instanceof UndoableCommand u) {
            undoStack.push(u);
            redoStack.clear();      // a new action destroys the "future" you could have redone
            System.out.println("  ran  " + command.name());
        } else {
            System.out.println("  ran  " + command.name() + " (irreversible: kept out of history)");
        }
    }

    public void undo() {
        if (undoStack.isEmpty()) { System.out.println("  nothing to undo"); return; }
        UndoableCommand c = undoStack.pop();
        c.undo();
        redoStack.push(c);
        System.out.println("  undo " + c.name());
    }

    public void redo() {
        if (redoStack.isEmpty()) { System.out.println("  nothing to redo"); return; }
        UndoableCommand c = redoStack.pop();
        c.execute();
        undoStack.push(c);
        System.out.println("  redo " + c.name());
    }
}
```

```java
// File: code/lld/ch10/command/Main.java
package lld.ch10.command;

import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Main {
    public static void main(String[] args) throws InterruptedException {
        Cart cart = new Cart();
        CommandHistory history = new CommandHistory();

        System.out.println("=== 1. Undo / redo ===");
        cart.setDiscount(5);   // the cart does NOT start at 0%: a guessing undo would be wrong
        history.run(new AddItemCommand(cart, "widget", 2));
        history.run(new AddItemCommand(cart, "gadget", 1));
        history.run(new ApplyDiscountCommand(cart, 10));
        System.out.println("  " + cart);
        history.undo();
        history.undo();
        System.out.println("  " + cart + "   <- discount restored to the recorded 5%, not 0%");
        history.redo();
        System.out.println("  " + cart);
        history.run(new AddItemCommand(cart, "cable", 3));   // clears the redo stack
        history.redo();

        System.out.println("\n=== 2. Macro command ===");
        UndoableCommand starterKit = new MacroCommand("starter-kit", List.of(
            new AddItemCommand(cart, "keyboard", 1),
            new AddItemCommand(cart, "mouse", 1),
            new ApplyDiscountCommand(cart, 15)));
        history.run(starterKit);
        System.out.println("  " + cart);
        history.undo();                                      // one undo reverses all three steps
        System.out.println("  " + cart);

        System.out.println("\n=== 3. Irreversible command (no undo offered) ===");
        Command sendReceipt = new Command() {
            @Override public void execute() { System.out.println("  [email] receipt sent for " + cart); }
            @Override public String name()  { return "SendReceipt"; }
        };
        history.run(sendReceipt);

        System.out.println("\n=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===");
        List<Runnable> jobs = List.of(
            () -> System.out.println("  [worker] resize image 1"),
            () -> System.out.println("  [worker] resize image 2"),
            () -> System.out.println("  [worker] send newsletter"));
        System.out.println("  queued " + jobs.size() + " jobs");
        ExecutorService worker = Executors.newSingleThreadExecutor();   // one thread => FIFO order
        jobs.forEach(worker::submit);
        worker.shutdown();
        worker.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("  all jobs done");
    }
}
```

Output:

```
=== 1. Undo / redo ===
  ran  AddItem(widget, 2)
  ran  AddItem(gadget, 1)
  ran  ApplyDiscount(10%)
  Cart[widget x2, gadget x1] discount=10%
  undo ApplyDiscount(10%)
  undo AddItem(gadget, 1)
  Cart[widget x2] discount=5%   <- discount restored to the recorded 5%, not 0%
  redo AddItem(gadget, 1)
  Cart[widget x2, gadget x1] discount=5%
  ran  AddItem(cable, 3)
  nothing to redo

=== 2. Macro command ===
  ran  Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3, keyboard x1, mouse x1] discount=15%
  undo Macro:starter-kit
  Cart[widget x2, gadget x1, cable x3] discount=5%

=== 3. Irreversible command (no undo offered) ===
  [email] receipt sent for Cart[widget x2, gadget x1, cable x3] discount=5%
  ran  SendReceipt (irreversible: kept out of history)

=== 4. Commands on a queue: Runnable + ExecutorService (the invoker) ===
  queued 3 jobs
  [worker] resize image 1
  [worker] resize image 2
  [worker] send newsletter
  all jobs done
```

Undo must restore, not recompute

It is tempting to write a discount undo as "reset to 0" or "calculate what the old value probably was". Both are **guesses**. If the cart began at 5%, or some other command touched the discount meanwhile, the user ends up with a value that never existed. The rule: **`execute` must record whatever `undo` will need** — normally just the fields it is about to overwrite, not a copy of the whole object.

Undo history is strictly last-in, first-out. Commands are reversed newest first: the sequence `AddItem, AddItem, ApplyDiscount` is undone as `ApplyDiscount`, then the second `AddItem`, then the first. Reversing them in any other order can produce states that never existed, hence a stack, never a list you can index into at random.

When undo is a lie. Some effects escape your system: an email that was sent, a card that was charged, a call made to a partner's API. An `undo()` on those promises something impossible, which is worse than having no undo at all. Two honest options: keep them out of the history (our `SendReceipt`), or use **compensation** — a *new* action that produces a similar end state, like a **refund** for a charge. A refund is not an undo: both events stay in the record.

Trade-offs

Pros	Cons
Since the request lives on after the call, undo/redo, queues, audit logs, retries and scheduling all become possible.	Every action needs its own class, so the number of classes tracks the number of actions.
The invoker has no knowledge of what it runs, so a single history works for all commands.	Each command has to record enough to run later and to reverse itself; subtle mistakes are easy.
One action object can back a menu entry, a hot-key, a script and an API endpoint.	Extra hop when reading code: to learn what a button does you first find the command it builds.
Commands can be grouped into macro commands.	A single faulty `undo` corrupts the whole history.

When not to use it

- **The call is simple and immediate.** A `SaveCommand` class that nobody ever undoes, queues or logs is extra code with no benefit.
- **Undo is not really possible.** Use compensation and call it that.
- **You only need deferral.** A lambda (`Runnable`) already is a one-method command. Promote it to a class once you need `undo`, a readable name in logs, or serialization.
- **Commands must outlive the process.** They then have to be serialized, which drags in format versioning; a durable queue or an event log (Kafka, a jobs table) handles that better than hand-rolled command classes.

Common mistakes

1. **Recomputing in `undo`** instead of restoring captured state.
2. **Forgetting to clear the redo stack** when a new command runs.
3. **Putting business logic in the invoker** (`if (cmd instanceof AddItem) ...`). The invoker should only run and remember.
4. **Commands that capture a mutable object by reference** and then read it later, when it has changed. Capture values.
5. **Unbounded history** – cap the depth, as editors do.

Real-world examples

- **Undo in editors**: word processors, IDEs, Photoshop. The JDK has `javax.swing.undo.UndoManager` with `UndoableEdit` objects that have `undo()` / `redo()`.
- **`Runnable` / `Callable`** are the JDK's command interfaces; `ExecutorService`, `ScheduledExecutorService` and `Thread` are invokers.
- **GUI actions**: Swing's `javax.swing.Action` lets one action back a menu item, a toolbar button and a shortcut; WPF's `ICommand` in .NET plays the same role.
- **Event sourcing** (Chapter 11): recording the *requested changes* instead of only the resulting state is the command idea applied to a whole system.
- **Job queues**: a Sidekiq/Celery job, a Spring `@Async` task, a message on a Kafka/RabbitMQ queue that asks a worker to "send invoice #42" – a serialized command.
- **Database migrations** with `up` / `down` (e.g. Liquibase changesets with rollback) mirror `execute` / `undo`.
- **Redux** actions are command objects: the store dispatches them (invoker role) and reducers apply them to state (receiver role); Redux DevTools' time travel replays them.
- **In other languages**: in Python, a callable object (one defining `__call__`) with an extra `undo` method works well, and `functools.partial` can bind the arguments; in C#, UI frameworks ship an `ICommand` interface, and a `record` gives you an immutable command in a single line.
- **Transactional outbox / task tables** in food-delivery or e-commerce back ends store "commands" (send SMS, notify restaurant) to be run reliably by workers.

In the interview

> **30-second answer:** "Command wraps a request, together with its receiver and arguments, in an object – so the request can be kept, queued, logged, retried or reversed. Three roles matter: the invoker triggers and records commands without knowing what they touch, each command can perform and reverse a single operation, and the receiver holds the state that actually changes. For undo, `execute` must capture the old value and `undo` restores it; history is a stack, and a new command clears redo. Irreversible actions like charging a card get compensation, not undo. In Java, `Runnable` plus an `ExecutorService` is the everyday form."

Follow-ups:

- *Command vs Strategy?* Each puts behaviour inside an object. A **Strategy** answers *how* a job is done and is picked and replaced by the caller. A **Command** captures *which* job to do, arguments included, and is held on to so it can run later or be reversed.
- *Command vs a plain lambda?* A one-method closure already is a command. Write a class only when you also need `undo`, a readable log name, or to serialize it.

- *Command vs Memento for undo?* Command stores the **action** plus a small piece of state; undo runs the inverse. Memento stores the **whole state**; undo puts a snapshot back. Command is cheaper per step; Memento is always correct. Real systems combine them (snapshot occasionally, replay commands in between).
- *How do you implement redo?* A second stack; undo moves a command to it; redo moves it back after re-executing; a new command clears it.
- *How do you make a command queue thread-safe?* Use a `BlockingQueue` or an `ExecutorService`; make commands immutable apart from captured undo state.
- *How would you persist commands?* Serialize them (JSON with a type field and a version), store in a durable queue or table, make them **idempotent** so a retry is safe.

Self-check

1. **Q:** What must a command carry? **A:** Its receiver and all arguments needed to run later (and any state needed for undo).
2. **Q:** Why does `ApplyDiscountCommand` store `previousPercent`? **A:** So undo restores the real previous value instead of guessing.
3. **Q:** What happens to the redo stack when a new command runs? **A:** It is cleared.
4. **Q:** How do you "undo" a card charge? **A:** You don't; you issue a compensating action (a refund) and keep both in the record.
5. **Q:** Which JDK interface is a command? **A:** `Runnable` (and `Callable`).

TL;DR

Make the request an object: **invoker** runs it, **command** knows do/undo, **receiver** changes. Undo **restores captured state**, history is a **stack**, and irreversible actions get **compensation**. A lambda is a command when you only need deferral.

10.3 Iterator

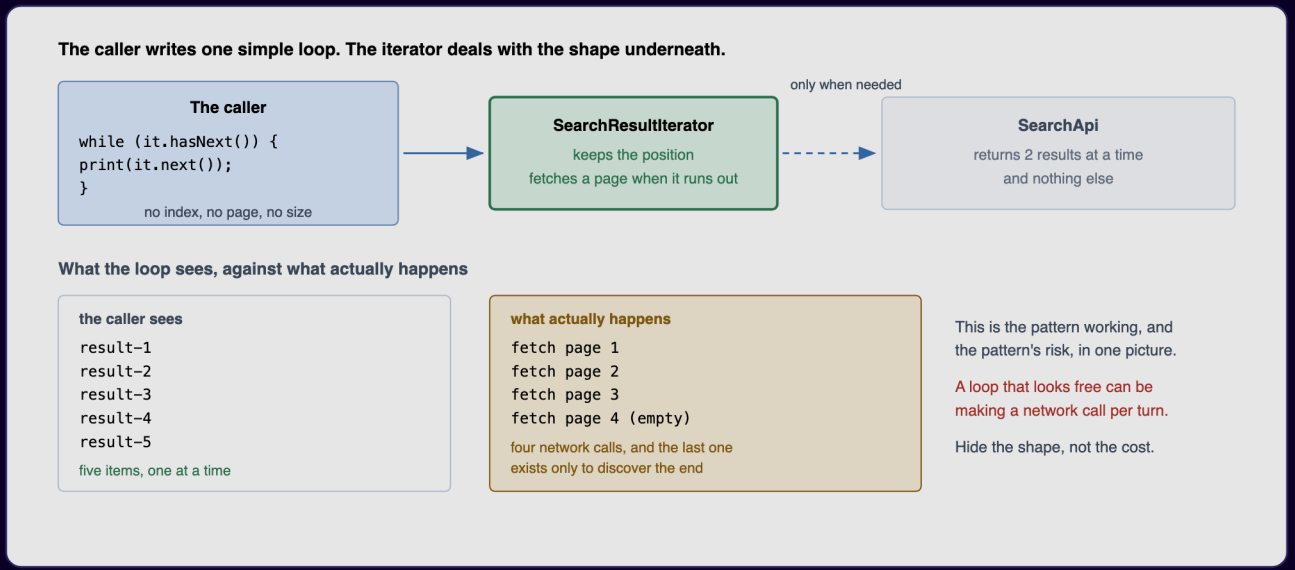


Figure – The caller writes one simple loop; the iterator fetches pages from the API only when needed (course diagram)

What it is

Iterator lets you visit the elements of a collection **one at a time, in order, without knowing how the collection stores them** – array, linked list, tree, or a remote API that returns two results per page.

GoF intent, in plain words: *give sequential access to the elements of an aggregate without exposing its internal representation.* GoF participants: **Iterator** (interface for accessing and traversing), **ConcreteIterator** (implements it and tracks the current position), **Aggregate** (interface for creating an iterator) and **ConcreteAggregate** (returns the right ConcreteIterator).

Everyday analogy: a TV remote's "next channel" button. You do not need to know how the channels are stored in the TV; you just press *next* until there are no more.

The problem it solves

- Code that loops with `get(i)` quietly assumes the data sits in an array. Move the data into a linked list, a tree or a paginated remote service, and each of those loops must be rewritten.
- Many sources **cannot be indexed at all**: a tree has several valid visiting orders (depth-first, breadth-first); a graph must remember which nodes were already seen; a search API hands out pages. When each caller writes its own traversal, each caller also re-implements paging — and some implement it wrongly.

Iterator moves the walking logic **into a separate object** that answers two questions: *are any elements left?* (`hasNext`) and *hand me the following element* (`next`).

Key detail: the current position is stored inside the iterator object, never in the collection. So two traversals of one collection can be in progress together without disturbing each other, and every call to `iterator()` must build a **new** iterator instead of rewinding a shared cursor.

How it works

1. The **aggregate** (the collection) has a method `iterator()` that creates a new iterator. In Java it implements `Iterable<T>`.
2. The **iterator** implements `Iterator<T>`: `hasNext()` and `next()` (and optionally `remove()`).
3. The iterator keeps its own **cursor** (index, stack of tree nodes, current page...).
4. The client loops: `while (it.hasNext()) use(it.next());` — or just `for (T x : collection)`, which the Java compiler turns into exactly that.

Structure (UML)

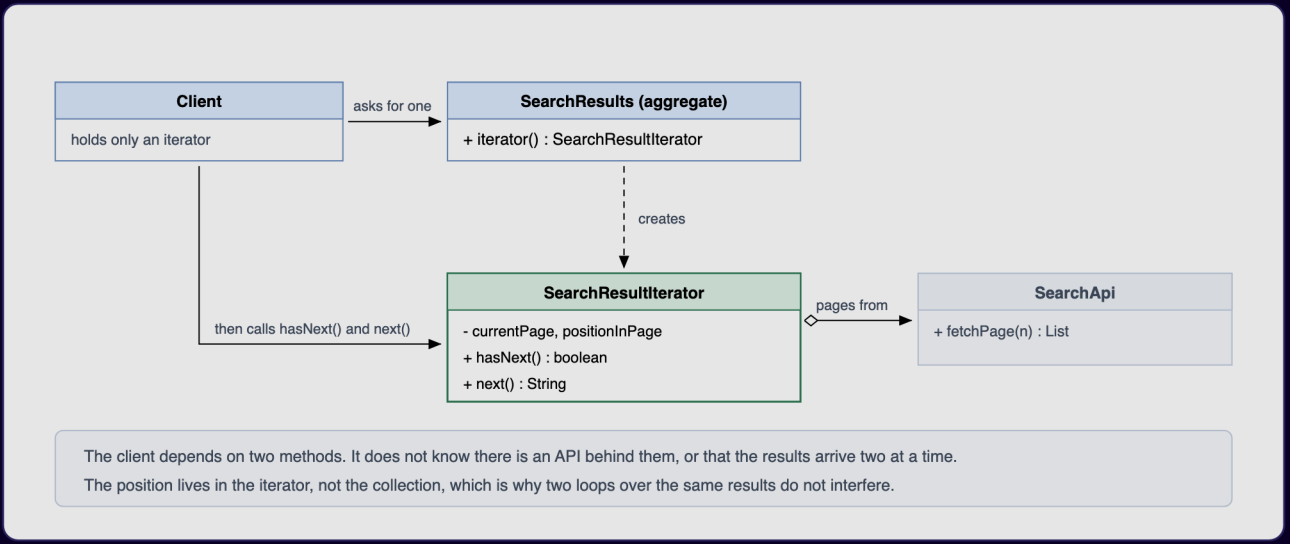
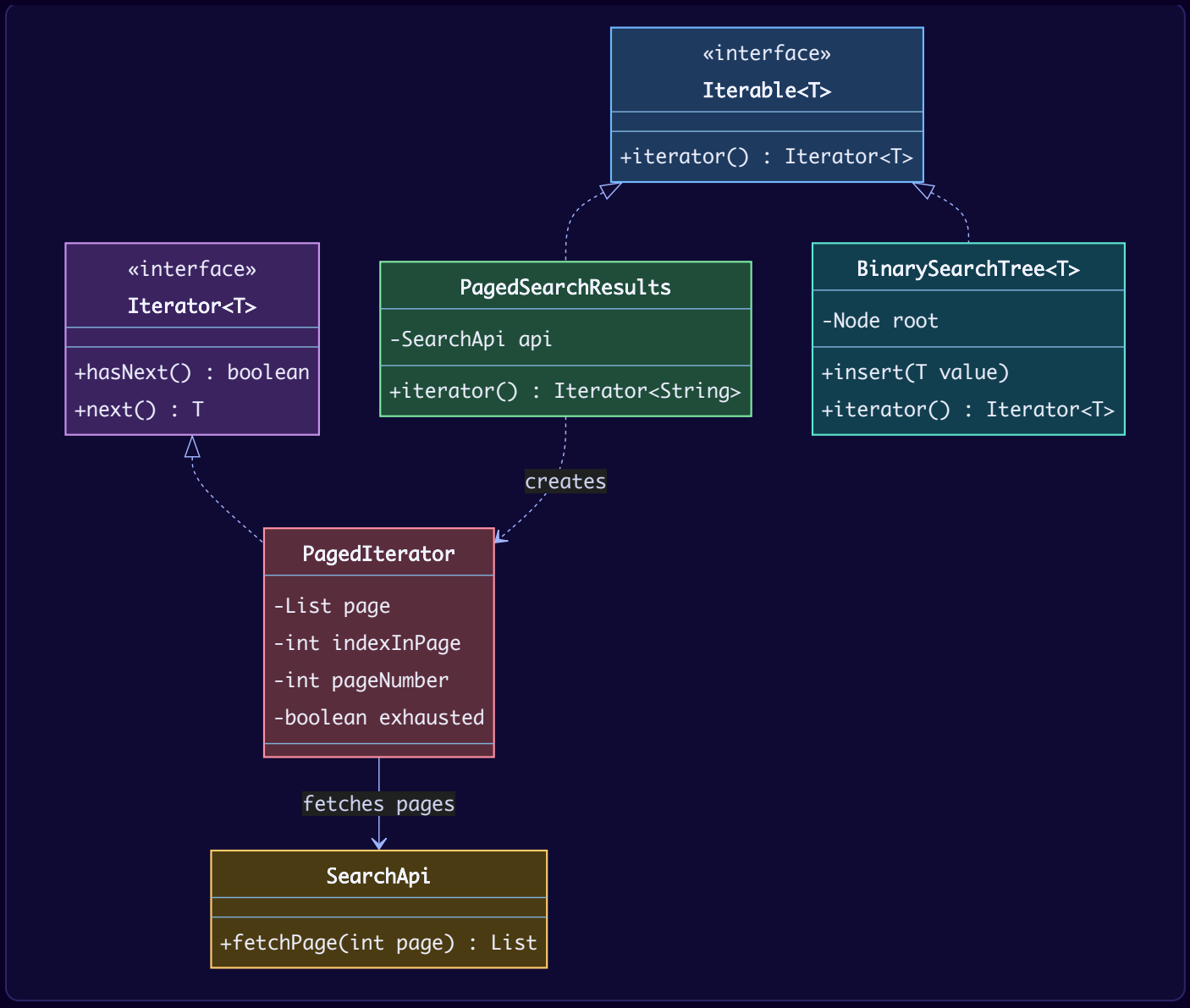


Figure – Class diagram: SearchResults (aggregate) hands out a SearchResultIterator, which calls SearchApi page by page; the client only calls hasNext/next (course diagram)

In the diagram above, `hasNext` does real work: once the buffered page runs out, it asks the API for another page. A *lazy* iterator (one that produces elements only on demand) commonly behaves like this — so never assume `hasNext` is cheap or has no side effects.

Our Java design:



Java example

`lld.ch10.iterator` shows:

1. **A paged remote source behind a plain for-each.** `PagedSearchResults implements Iterable<String>`, so `for (String r : results)` just works. The inner `PagedIterator` fetches a page only when the current one is used up, and follows the `Iterator` contract (`next()` throws `NoSuchElementException` when empty).
2. **A tree iterator.** `BinarySearchTree` hides an in-order walk behind `hasNext/next` using an explicit stack; two iterators over the same tree run independently.
3. **External vs internal iteration.** *External*: the **client** drives (`hasNext/next`) and can stop early with `break`. *Internal*: the **collection** drives and calls your code for each element (`forEach(lambda)`, streams).
4. **Fail-fast vs fail-safe.** Structurally changing an `ArrayList` (adding or removing) while a for-each loop is walking it *usually* makes the next `next()` throw `ConcurrentModificationException`. It is only **best-effort**: the demo also removes the second-to-last element, after which `hasNext()` simply returns `false` – no exception, and the last element is silently never visited. The safe way is `Iterator.remove()` (or `removeIf`). `CopyOnWriteArrayList` iterates over a **snapshot**, so it never throws – but the loop does not see the new elements. ("Fail-safe" is an informal name; the Javadoc talks about *snapshot* and *weakly consistent* iterators.)
5. `Spliterator` (Java 8): an iterator that can **split** itself (`trySplit`) so parallel streams can process parts on different threads. It reports **characteristics** such as `SIZED` and `ORDERED`.

```java
// File: code/lld/ch10/iterator/SearchApi.java
package lld.ch10.iterator;

import java.util.List;

/** A remote source that returns two results per page and knows nothing about iteration. */
final class SearchApi {
    private static final List<String> ALL = List.of("result-1", "result-2", "result-3", "result-4", "result-
5");
    private static final int PAGE_SIZE = 2;
    private int calls = 0;

    List<String> fetchPage(int page) {          // pages are 1-based
        calls++;
        System.out.println("    [api] fetching page " + page);
        int from = (page - 1) * PAGE_SIZE;
        if (from >= ALL.size()) return List.of();
        return ALL.subList(from, Math.min(from + PAGE_SIZE, ALL.size()));
    }

    int calls() { return calls; }
}
```

```java
// File: code/lld/ch10/iterator/PagedSearchResults.java
package lld.ch10.iterator;

import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

/**
 * The AGGREGATE. Implements Iterable, so the enhanced for-loop works on it.
 * Its only job is to hand out a FRESH iterator each time.
 */
final class PagedSearchResults implements Iterable<String> {
    private final SearchApi api;

    PagedSearchResults(SearchApi api) { this.api = api; }

    @Override
    public Iterator<String> iterator() { return new PagedIterator(); }

    /** The ITERATOR. The position lives here, not in the collection. */
    private final class PagedIterator implements Iterator<String> {
        private List<String> page = List.of();
        private int indexInPage = 0;
        private int pageNumber = 0;
        private boolean exhausted = false;

        @Override
        public boolean hasNext() {                // NOT a passive question: may hit the network
            if (indexInPage < page.size()) return true;
            if (exhausted) return false;
            page = api.fetchPage(++pageNumber);
            indexInPage = 0;
            if (page.isEmpty()) {
                exhausted = true;
                return false;
            }
            return true;
        }

        @Override
        public String next() {
            if (!hasNext()) throw new NoSuchElementException();   // the Iterator contract
            return page.get(indexInPage++);
        }
    }
}
```

```java
// File: code/lld/ch10/iterator/BinarySearchTree.java
package lld.ch10.iterator;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Iterator;
import java.util.NoSuchElementException;

/** A tree has no index. The iterator hides an in-order walk behind hasNext/next. */
final class BinarySearchTree<T extends Comparable<T>> implements Iterable<T> {
    private static final class Node<T> {
        final T value;
        Node<T> left, right;
        Node(T value) { this.value = value; }
    }

    private Node<T> root;

    void insert(T value) {
        Node<T> fresh = new Node<>(value);
        if (root == null) { root = fresh; return; }
        Node<T> cur = root;
        while (true) {
            if (value.compareTo(cur.value) < 0) {
                if (cur.left == null) { cur.left = fresh; return; }
                cur = cur.left;
            } else {
                if (cur.right == null) { cur.right = fresh; return; }
                cur = cur.right;
            }
        }
    }

    @Override
    public Iterator<T> iterator() {
        return new Iterator<>() {
            // The stack holds the path still to visit: this IS the iterator's position.
            private final Deque<Node<T>> stack = new ArrayDeque<>();
            { pushLeft(root); }

            private void pushLeft(Node<T> n) {
                for (; n != null; n = n.left) stack.push(n);
            }

            @Override public boolean hasNext() { return !stack.isEmpty(); }

            @Override
            public T next() {
                if (stack.isEmpty()) throw new NoSuchElementException();
                Node<T> n = stack.pop();
                pushLeft(n.right);
                return n.value;
            }
        };
    }
}
```

```java
// File: code/lld/ch10/iterator/Main.java
package lld.ch10.iterator;

import java.util.ArrayList;
import java.util.ConcurrentModificationException;
import java.util.Iterator;
import java.util.List;
import java.util.Spliterator;
import java.util.concurrent.CopyOnWriteArrayList;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Paged remote results behind a plain for-each ===");
        SearchApi api = new SearchApi();
        for (String r : new PagedSearchResults(api)) {
            System.out.println("  loop sees " + r);
        }
        System.out.println("  5 items, " + api.calls() + " network calls (the last one only discovers the
end)");

        System.out.println("\n=== 2. Two independent iterators over one tree ===");
        BinarySearchTree<Integer> tree = new BinarySearchTree<>();
        for (int v : new int[] {50, 30, 70, 20, 40, 60, 80}) tree.insert(v);
        Iterator<Integer> a = tree.iterator();
        Iterator<Integer> b = tree.iterator();
        System.out.println("  a: " + a.next() + ", " + a.next() + ", " + a.next());
        System.out.println("  b: " + b.next() + "   <- b starts from the beginning: position lives in the
iterator");
        StringBuilder all = new StringBuilder();
        for (int v : tree) all.append(v).append(' ');
        System.out.println("  in-order: " + all.toString().trim());

        System.out.println("\n=== 3. External vs internal iteration ===");
        List<String> names = List.of("ann", "bob", "cy");
        Iterator<String> it = names.iterator();                // external: the CLIENT drives
        while (it.hasNext()) {
            String n = it.next();
            if (n.equals("bob")) { System.out.println("  external: found bob, stopping early"); break; }
        }
        names.forEach(n -> System.out.println("  internal: " + n)); // internal: the COLLECTION drives

        System.out.println("\n=== 4. Fail-fast vs fail-safe ===");
        List<String> tags = new ArrayList<>(List.of("java", "tmp", "lld", "tmp"));
        try {
            for (String t : tags) {
                if (t.equals("tmp")) tags.remove(t);           // structural change behind the iterator's back
            }
        } catch (ConcurrentModificationException e) {
            System.out.println("  fail-fast: ConcurrentModificationException");
        }
        List<String> quiet = new ArrayList<>(List.of("java", "tmp", "lld"));
        List<String> visited = new ArrayList<>();
        for (String t : quiet) {
            visited.add(t);
            if (t.equals("tmp")) quiet.remove(t);              // removes the SECOND-TO-LAST element
        }
        System.out.println("  best-effort only: no exception, visited " + visited + ", 'lld' silently
skipped");
        tags = new ArrayList<>(List.of("java", "tmp", "lld", "tmp"));
        for (Iterator<String> i = tags.iterator(); i.hasNext(); ) {
            if (i.next().equals("tmp")) i.remove();            // the safe way: remove through the iterator
        }
        System.out.println("  after Iterator.remove: " + tags);
        List<String> cow = new CopyOnWriteArrayList<>(List.of("x", "y"));
        for (String s : cow) cow.add(s + "!");                  // iterates a snapshot: no exception
        System.out.println("  fail-safe CopyOnWriteArrayList: " + cow);
```

```
        System.out.println("\n=== 5. Spliterator: an iterator that can split for parallel work ===");
        List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8);
        Spliterator<Integer> right = nums.spliterator();
        Spliterator<Integer> left = right.trySplit();           // hands off (roughly) the first half
        System.out.println("  SIZED=" + right.hasCharacteristics(Spliterator.SIZED)
                + " ORDERED=" + right.hasCharacteristics(Spliterator.ORDERED));
        StringBuilder l = new StringBuilder(), r = new StringBuilder();
        left.forEachRemaining(x -> l.append(x).append(' '));
        right.forEachRemaining(x -> r.append(x).append(' '));
        System.out.println("  left half:  " + l.toString().trim());
        System.out.println("  right half: " + r.toString().trim());
        System.out.println("  parallel stream sum = " +
nums.parallelStream().mapToInt(Integer::intValue).sum());
    }
}
```

Output:

```
=== 1. Paged remote results behind a plain for-each ===
    [api] fetching page 1
  loop sees result-1
  loop sees result-2
    [api] fetching page 2
  loop sees result-3
  loop sees result-4
    [api] fetching page 3
  loop sees result-5
    [api] fetching page 4
  5 items, 4 network calls (the last one only discovers the end)

=== 2. Two independent iterators over one tree ===
  a: 20, 30, 40
  b: 20   <- b starts from the beginning: position lives in the iterator
  in-order: 20 30 40 50 60 70 80

=== 3. External vs internal iteration ===
  external: found bob, stopping early
  internal: ann
  internal: bob
  internal: cy

=== 4. Fail-fast vs fail-safe ===
  fail-fast: ConcurrentModificationException
  best-effort only: no exception, visited [java, tmp], 'lld' silently skipped
  after Iterator.remove: [java, lld]
  fail-safe CopyOnWriteArrayList: [x, y, x!, y!]

=== 5. Spliterator: an iterator that can split for parallel work ===
  SIZED=true ORDERED=true
  left half:  1 2 3 4
  right half: 5 6 7 8
  parallel stream sum = 36
```

Hide the shape, not the cost

Look at section 1 of the output: five results, **four** network calls. The last call comes back empty; its only purpose is to find out that the data has ended, because this API reports neither a total nor a "next page" token. That is genuine behaviour, not a bug in the demo. The takeaway: **a tidy loop conceals how much each step costs.** The **N+1 query problem** in ORMs has the same shape (one query for the list, then one more query *per element* as you iterate). Ways to keep the cost visible:

- **Name the type honestly** (`PagedResults`, `LazySearchResults`, not `List`).
- **Offer a batch API** for callers who want everything, with bigger pages.
- **Return the total or a next-page cursor** when the source knows it, so the extra call is not needed.

- **Log or count the fetches**, so a growing request count gets noticed.

Java's iterator family (quick reference)

Type	What it adds
`Iterable<T>`	`iterator()`; lets a type be used in the enhanced for-loop. Also `forEach` (internal iteration) and `spliterator()`.
`Iterator<T>`	`hasNext()`, `next()`, optional `remove()`, `forEachRemaining()`.
`ListIterator<T>`	Bidirectional: `hasPrevious()`, `previous()`, indexes, `set()`, `add()`.
`Spliterator<T>`	`tryAdvance`, `forEachRemaining`, `trySplit` for parallelism, `characteristics()` like `ORDERED`, `SIZED`, `DISTINCT`, `IMMUTABLE`.
`Enumeration<E>`	The old (Java 1.0) version: `hasMoreElements()`, `nextElement()`. Prefer `Iterator`.

Fail-fast, weakly consistent, snapshot. Iterators of `ArrayList`, `HashMap` and friends are **fail-fast**: they detect a structural change made behind their back and throw `ConcurrentModificationException`. The Javadoc says this is done on a **best-effort** basis – never write code that relies on the exception (section 4 of the demo shows a removal that goes unnoticed). Despite its name, the exception is most often caused by **one thread** changing a collection inside its own loop, not by concurrency. `ConcurrentHashMap` iterators are **weakly consistent** (never throw, may or may not show concurrent changes). `CopyOnWriteArrayList` iterators work on a **snapshot** (and their `remove()` throws `UnsupportedOperationException`).

Trade-offs

Pros	Cons
Client code no longer cares whether data lives in an array, in a tree or behind an API.	One more object is created per traversal (only noticeable in extremely hot loops).
One structure can offer several named, reusable visiting orders.	Changing the collection mid-loop is a bug unless you use the iterator's own `remove` or a concurrent collection.
Elements can be produced lazily, so even huge or endless sequences work.	If a lazy iterator owns a file or DB cursor and the loop breaks early, the resource leaks unless it is closed.
Paging, buffering and prefetching disappear behind an ordinary loop.	Per-step cost becomes invisible – the root of N+1 problems.

When not to use it

- **The data is already an in-memory list.** Hand out the list (or its own iterator); writing a custom iterator around an array adds code and no value.
- **Callers jump around** ("I need element 4,000"). Iterators only move forward step by step; offer indexed access instead.
- **All elements are in memory and always consumed in full.** An ordinary loop over the list is simpler and faster.
- **A step has side effects the caller must decide on.** Silently loading another page is acceptable; silently charging a customer is not.

Common mistakes

1. **Modifying the collection while iterating** (other than through `Iterator.remove`). Collect changes and apply them after the loop, or use `removeIf`.
2. **Leaking a resource** from a lazy iterator on early `break`. Use try-with-resources (`Files.lines`, `DirectoryStream` and JDBC `ResultSet` are all closeable).
3. **Reusing a one-shot iterator** or stream a second time: it is already exhausted. A Java `Stream` at least throws `IllegalStateException`; a Python generator just yields nothing the second time, so the bug shows up as missing data. That is why `iterator()` must return a *new* object.

4. `next()` without `hasNext()`, or a `next()` that returns `null` at the end instead of throwing `NoSuchElementException`.
5. **Storing the cursor in the collection** – then two loops interfere.
6. **Assuming every language fails loudly.** Java's fail-fast iterators usually throw `ConcurrentModificationException` when you modify a list mid-loop; Python may simply skip elements and finish with a wrong result. Record the intended changes while looping and make them once the loop has finished.

Real-world examples

- **Every for-each loop** in Java, Python, C#, JavaScript (`for...of`).
- **JDBC** `ResultSet` moves through rows that need not all be loaded at once; `Scanner` implements `Iterator<String>`; `Files.newDirectoryStream` returns an `Iterable` over directory entries.
- **Paginated SDKs**: the AWS SDK for Java v2 "paginators" and Stripe's auto-pagination give you an iterable that fetches pages behind the scenes – exactly section 1.
- **Java** `Stream`**s** are built on `Spliterator`s; that is how parallel streams split work. C#'s LINQ is likewise a lazy layer over iteration, which is why both can work on endless sequences.
- **Generators**: Python's `yield`, JavaScript's `function*` and C#'s `yield return` let you write an iterator as a *function*; the language remembers the position for you. A generator is the same pattern with less code.
- **Kafka consumer** `poll()` **loops**, and **infinite feeds** (Instagram/Twitter timelines) that load the next page as you scroll.

In the interview

> **30-second answer:** "Iterator pulls traversal out into a separate object with `hasNext` and `next`, so a caller can walk data without caring whether the data is an array, a tree or a paginated API. Each iterator keeps its own position, so two loops over the same data are independent. Rather than a home-made interface I'd implement the language's protocol – `Iterable<T>` in Java – so the native for-each works. My example would be the lazy paged case, not an array, and I'd flag the risk: when every step can hit the network, the loop hides its cost – that is how N+1 problems start."

Follow-ups:

- *Internal vs external iterator?* External: the client controls stepping (`hasNext/next`) and can stop, pause or interleave two iterators. Internal: the collection controls the loop and calls your function (`forEach`, streams); simpler, easier to parallelise, but harder to stop early or to walk two collections in lock-step.
- *What is fail-fast?* The iterator checks a modification counter and throws `ConcurrentModificationException` if the collection was structurally changed outside the iterator. Best-effort only.
- *How do you remove while iterating?* `Iterator.remove()` or `Collection.removeIf(predicate)`.
- *What is a Spliterator?* A splittable iterator used by streams; `trySplit` hands off part of the elements so another thread can process them; characteristics tell the stream framework what it can optimise.
- *Iterator vs Visitor?* Iterator decides **how you move** through a structure; Visitor decides **what you do** at each element. They are often used together.
- *Iterator vs Composite?* Composite makes leaves and branches of a tree look alike; an iterator supplies a way to visit them. The two are often used together.
- *Iterator vs generator?* A generator (Python `yield`, JS `function*`) is the same pattern with the class replaced by a function that `yield`s.
- *Iterator vs stream?* Streams sit on top of iteration and add operations such as `map` and `filter`. When you need to transform data, not merely visit it, reach for streams.
- *How is Iterator expressed in other languages?* Python: `__iter__` / `__next__` or a generator. JavaScript/TypeScript: `[Symbol.iterator]` or `function*`, which makes `for...of` and spread work. C#: `IEnumerable<T>` with `yield return`. C++: `begin()` / `end()` for range-based for and the standard algorithms. Go: traditionally a `Next() bool` plus a value accessor (`sql.Rows`, `bufio.Scanner`), and since Go 1.23 range-over-function iterators.

Self-check

1. **Q:** Where does the cursor live? **A:** In the iterator object.
2. **Q:** What does the enhanced for-loop need from a class? **A:** That it implements `Iterable<T>`.
3. **Q:** Why did five results take four API calls? **A:** Without a total or cursor, the iterator can only discover the end by asking for one more page.
4. **Q:** Does `CopyOnWriteArrayList`'s iterator throw on concurrent modification? **A:** No; it iterates over a snapshot taken when the iterator was created.
5. **Q:** What should `next()` do when there are no more elements? **A:** Throw `NoSuchElementException`.

TL;DR

A separate object walks the collection with `hasNext/next`; the **position lives in the iterator**. Implement `Iterable` in Java. Know **fail-fast** vs snapshot iterators, **internal vs external** iteration, and don't let a lazy loop **hide its cost**.

10.4 Mediator

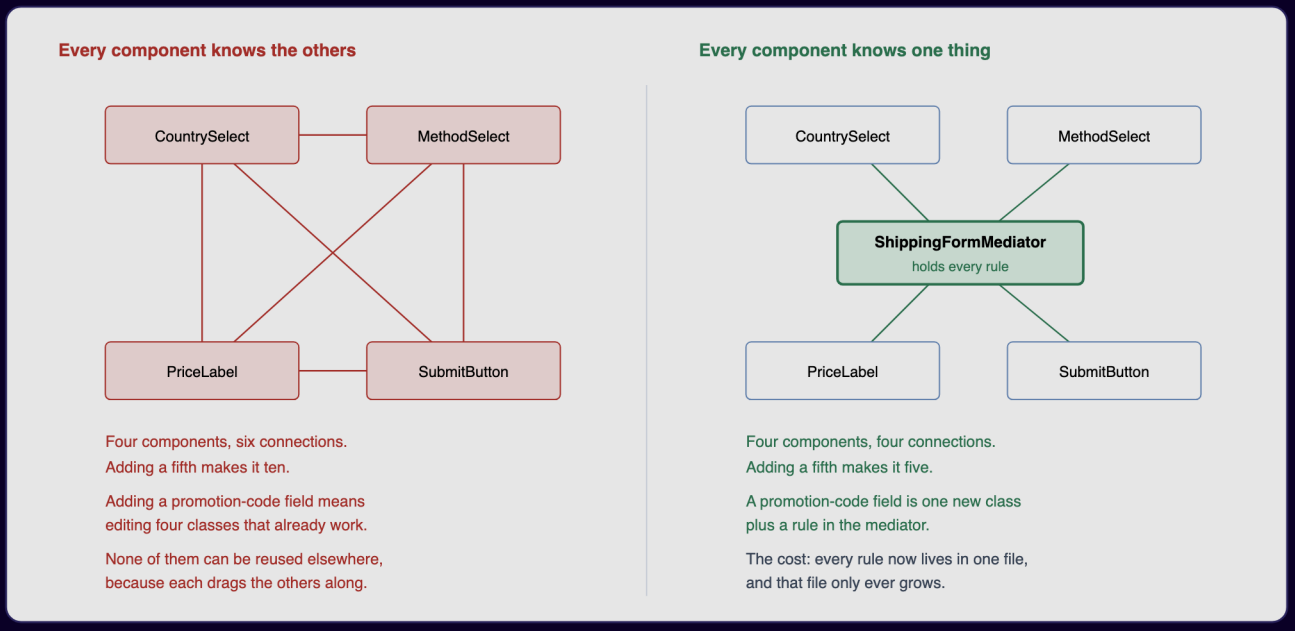


Figure – Left: four form components all know each other (six links). Right: each knows only the mediator (four links) (course diagram)

What it is

Mediator makes a group of objects talk through **a single coordinator** instead of holding references to one another. Each object only reports "this just happened" to the mediator, and the mediator works out how the rest should respond.

GoF intent, in plain words: *define an object that encapsulates how a set of objects interact, so they stop referring to each other explicitly and their interaction can be varied independently.* GoF participants: **Mediator** (interface for talking to colleagues), **ConcreteMediator** (knows the colleagues and implements the coordination), and **Colleague** classes (each knows its mediator and talks to it instead of to other colleagues). Some literature simply calls the mediator a *controller*.

Everyday analogy: **air traffic control**. Planes do not radio each other to agree who lands first. Each plane talks only to the tower, and the tower holds the rules about spacing and order.

The problem it solves

Picture a checkout form with four widgets: a **country** picker, a **delivery-method** picker, a **price** label and a **submit** button. Picking a country refreshes the list of delivery methods; the new method changes the price; the price situation decides whether submit is clickable. For some destinations no delivery is offered at all.

If each widget talks to the others directly:

- every widget holds references to three others — four widgets need **six** connections, five need **ten** (n(n−1)/2, growing roughly with the square);
- a new "promo code" field forces changes to four classes that were already tested and working;
- no widget can be reused on another screen: if `CountrySelect` calls `methodSelect.reload()`, it cannot be used anywhere without bringing `MethodSelect` too.

With a mediator, each widget has **one** reference (to the mediator): four widgets, four links; a fifth makes five. `CountrySelect` is now unaware that any price label exists, so another screen can reuse it with its own mediator and its own rules. Each widget also has no idea how many other widgets exist.

The honest trade: the dependencies are not removed — **they are relocated** into the mediator, a class that tends to keep getting bigger.

How it works

1. Define a **Mediator** interface, e.g. `notify(sender, event)`.
2. Each **component** (colleague) holds a reference to the mediator — and nothing else.
3. When something happens, a component calls `mediator.notify(this, event)`.
4. The **concrete mediator** holds all components and all the rules: "when the country changes, reload methods, then show the price, then enable or disable submit".
5. The mediator calls back **down** into specific components. Traffic is **two-way** through one object.

Structure (UML)

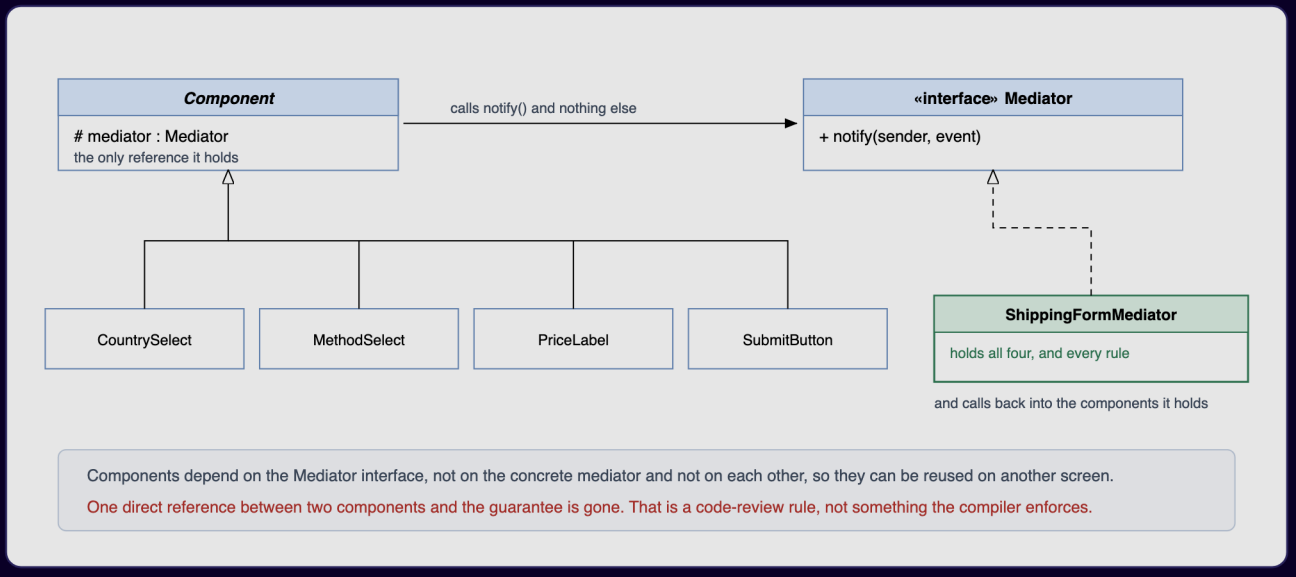
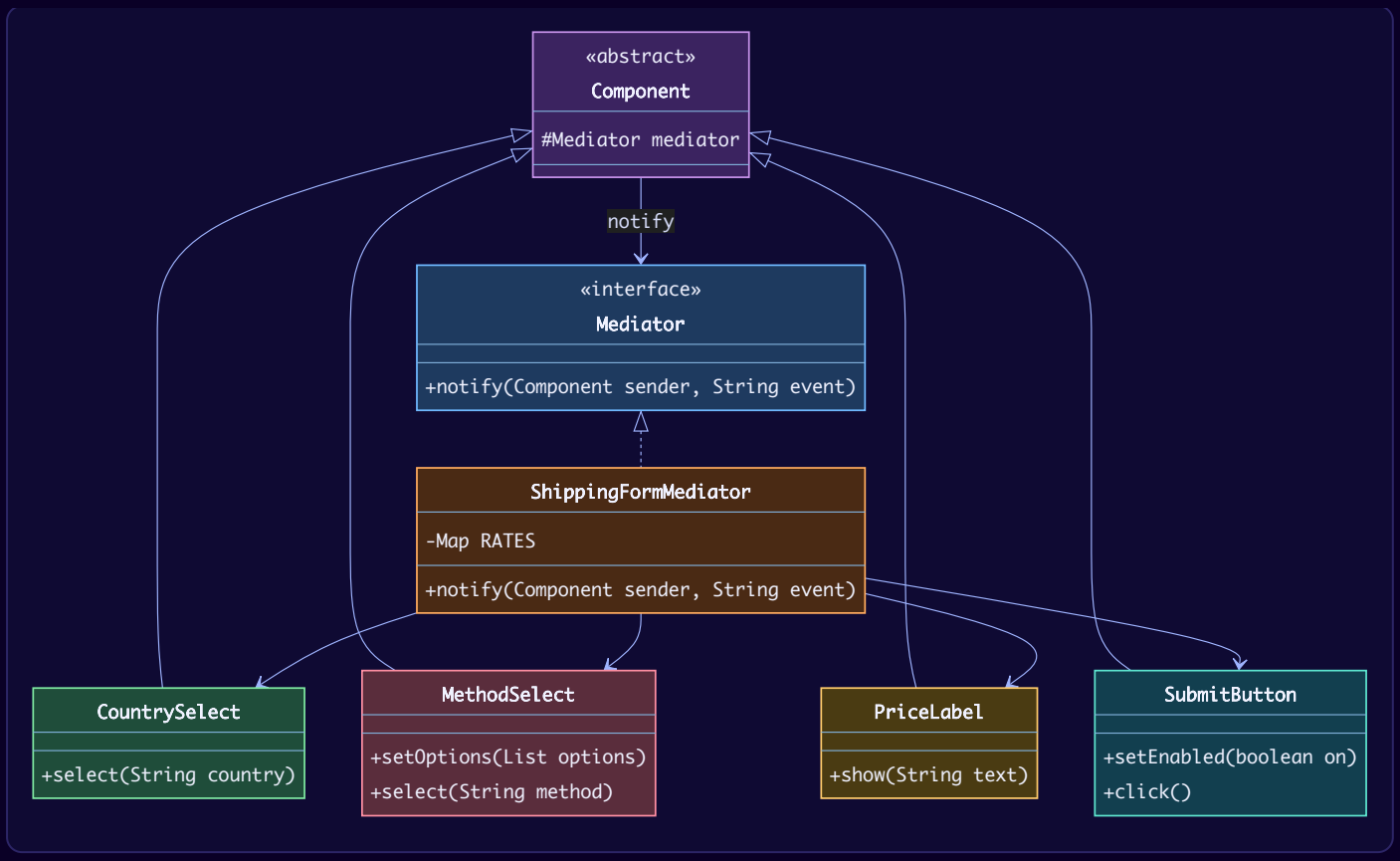
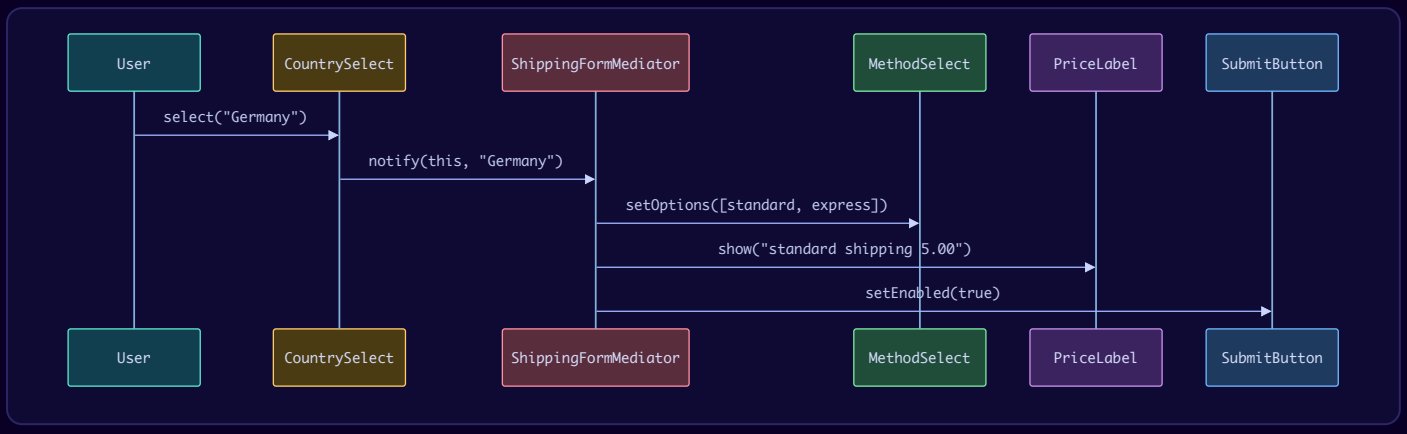


Figure — Class diagram: components extend Component (which holds a Mediator); ShippingFormMediator implements Mediator and holds all four components (course diagram)

In the diagram above, follow the arrows: widgets report **upward** by calling `notify`, and the mediator issues instructions **downward** to the widgets it owns. Traffic in both directions passing through one hub is what separates a mediator from an observer.

Our Java design:





Java example

`lld.ch10.mediator` has three mediators:

1. **Shipping form** (`ShippingFormMediator` + four components). No code except `notify` knows that choosing a country influences the price. Components identify themselves by reference (`sender == country`), not by strings. Shipping rates are `BigDecimal` — money is never stored as `double`.
2. **Chat room** (`ChatRoom`, `ChatUser`). Users never hold references to each other. The room routes broadcasts and private messages and applies **moderation** rules — rules live in the mediator, not in the users.
3. **Air traffic control** (`ControlTower`, `Flight`). Flights only request landing and report "vacated"; the tower owns the runway and the holding queue.

```java
// File: code/lld/ch10/mediator/Mediator.java
package lld.ch10.mediator;

/** Components report "something happened" here. They depend on this interface only. */
public interface Mediator {
    void notify(Component sender, String event);
}
```

```java
// File: code/lld/ch10/mediator/Component.java
package lld.ch10.mediator;

/** Base for every form widget: its ONLY reference is the mediator. */
public abstract class Component {
    protected Mediator mediator;

    void setMediator(Mediator mediator) { this.mediator = mediator; }
}
```

```java
// File: code/lld/ch10/mediator/CountrySelect.java
package lld.ch10.mediator;

final class CountrySelect extends Component {
    private String selected;

    void select(String country) {
        selected = country;
        System.out.println("  [country] user picked " + country);
        mediator.notify(this, country);          // report up; never call a sibling
    }

    String selected() { return selected; }
}
```

```java
// File: code/lld/ch10/mediator/MethodSelect.java
package lld.ch10.mediator;

import java.util.List;

final class MethodSelect extends Component {
    private List<String> options = List.of();
    private String selected;

    void setOptions(List<String> options) {
        this.options = options;
        this.selected = options.isEmpty() ? null : options.get(0);
        System.out.println("  [methods] reloaded: " + options + (selected == null ? "" : ", default " +
selected));
    }

    void select(String method) {
        if (!options.contains(method)) throw new IllegalArgumentException(method + " not offered");
        selected = method;
        System.out.println("  [methods] user picked " + method);
        mediator.notify(this, method);
    }

    String selected() { return selected; }
}
```

```java
// File: code/lld/ch10/mediator/PriceLabel.java
package lld.ch10.mediator;

final class PriceLabel extends Component {
    void show(String text) { System.out.println("  [price] " + text); }
}
```

```java
// File: code/lld/ch10/mediator/SubmitButton.java
package lld.ch10.mediator;

final class SubmitButton extends Component {
    private boolean enabled;

    void setEnabled(boolean enabled) {
        this.enabled = enabled;
        System.out.println("  [submit] " + (enabled ? "enabled" : "disabled"));
    }

    void click() {
        if (enabled) mediator.notify(this, "click");
        else System.out.println("  [submit] click ignored: button is disabled");
    }
}
```

```java
// File: code/lld/ch10/mediator/ShippingFormMediator.java
package lld.ch10.mediator;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.Map;

/** The ONLY place that knows how a country change affects methods, price and submit. */
final class ShippingFormMediator implements Mediator {
    private static final Map<String, Map<String, BigDecimal>> RATES = Map.of(
        "Germany", ordered("standard", "5.00", "express", "15.00"),
        "USA", ordered("standard", "8.00", "express", "20.00"),
        "Antarctica", Map.of());

    private final CountrySelect country;
    private final MethodSelect methods;
    private final PriceLabel price;
    private final SubmitButton submit;

    ShippingFormMediator(CountrySelect c, MethodSelect m, PriceLabel p, SubmitButton s) {
        this.country = c; this.methods = m; this.price = p; this.submit = s;
        for (Component comp : new Component[] {c, m, p, s}) comp.setMediator(this);
        submit.setEnabled(false);
    }

    @Override
    public void notify(Component sender, String event) {
        if (sender == country) {
            Map<String, BigDecimal> available = RATES.getOrDefault(event, Map.of());
            methods.setOptions(new ArrayList<>(available.keySet()));
            if (available.isEmpty()) {
                price.show("shipping not available");
                submit.setEnabled(false);
            } else {
                showPrice();                          // ORDER matters: reload methods first, then price
                submit.setEnabled(true);
            }
        } else if (sender == methods) {
            showPrice();
        } else if (sender == submit) {
            System.out.println("  [mediator] order placed: " + country.selected() + " / " +
methods.selected());
        }
    }

    private void showPrice() {
        BigDecimal p = RATES.get(country.selected()).get(methods.selected());   // money: BigDecimal, never
double
        price.show(methods.selected() + " shipping " + p.toPlainString());
    }

    private static Map<String, BigDecimal> ordered(String k1, String v1, String k2, String v2) {
        Map<String, BigDecimal> m = new LinkedHashMap<>();   // keeps "standard" before "express"
        m.put(k1, new BigDecimal(v1));
        m.put(k2, new BigDecimal(v2));
        return m;
    }
}
```

```java
// File: code/lld/ch10/mediator/ChatRoom.java
package lld.ch10.mediator;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;

/** Chat mediator: users never hold references to each other, only to the room. */
final class ChatRoom {
    private final Map<String, ChatUser> members = new LinkedHashMap<>();
    private final Set<String> bannedWords = Set.of("spam");

    void join(ChatUser user) {
        members.put(user.name(), user);
        user.joined(this);
        broadcast("room", user.name() + " joined");
    }

    /** Room rules (moderation, routing) live here, not in the users. */
    void send(ChatUser from, String to, String text) {
        if (bannedWords.stream().anyMatch(text.toLowerCase()::contains)) {
            from.receive("room", "message blocked by moderation");
            return;
        }
        if (to == null) {
            members.values().stream().filter(u -> u != from).forEach(u -> u.receive(from.name(), text));
        } else if (members.containsKey(to)) {
            members.get(to).receive(from.name() + " (private)", text);
        } else {
            from.receive("room", to + " is not in the room");
        }
    }

    private void broadcast(String from, String text) {
        members.values().forEach(u -> u.receive(from, text));
    }
}
```

```java
// File: code/lld/ch10/mediator/ChatUser.java
package lld.ch10.mediator;

final class ChatUser {
    private final String name;
    private ChatRoom room;

    ChatUser(String name) { this.name = name; }

    String name() { return name; }
    void joined(ChatRoom room) { this.room = room; }
    void say(String text) { room.send(this, null, text); }
    void whisper(String to, String text) { room.send(this, to, text); }
    void receive(String from, String text) { System.out.println("  " + name + " sees <" + from + "> " + text);
}
}
```

```java
// File: code/lld/ch10/mediator/ControlTower.java
package lld.ch10.mediator;

import java.util.ArrayDeque;
import java.util.Queue;

/** Air traffic control: aircraft talk to the tower, never to each other. */
final class ControlTower {
    private Flight onRunway;                          // null = runway free
    private final Queue<Flight> holding = new ArrayDeque<>();

    void requestLanding(Flight f) {
        if (onRunway == null) {
            clear(f);
        } else {
            holding.add(f);
            System.out.println("  [tower] " + f.callsign() + ": hold, runway busy with " +
onRunway.callsign());
        }
    }

    void runwayVacated(Flight f) {
        System.out.println("  [tower] " + f.callsign() + " vacated the runway");
        onRunway = null;
        Flight next = holding.poll();
        if (next != null) clear(next);
    }

    private void clear(Flight f) {
        onRunway = f;
        System.out.println("  [tower] " + f.callsign() + ": cleared to land");
    }
}
```

```java
// File: code/lld/ch10/mediator/Flight.java
package lld.ch10.mediator;

record Flight(String callsign, ControlTower tower) {
    void requestLanding() { tower.requestLanding(this); }
    void landed() { tower.runwayVacated(this); }
}
```

```java
// File: code/lld/ch10/mediator/Main.java
package lld.ch10.mediator;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Shipping form: widgets coordinated by a mediator ===");
        CountrySelect country = new CountrySelect();
        MethodSelect methods = new MethodSelect();
        PriceLabel price = new PriceLabel();
        SubmitButton submit = new SubmitButton();
        new ShippingFormMediator(country, methods, price, submit);

        country.select("Germany");        // reloads methods, shows price, enables submit
        methods.select("express");        // updates price only
        submit.click();
        country.select("Antarctica");     // no methods: price hidden, submit disabled
        submit.click();

        System.out.println("\n=== 2. Chat room ===");
        ChatRoom room = new ChatRoom();
        ChatUser asha = new ChatUser("asha"), ben = new ChatUser("ben"), chen = new ChatUser("chen");
        room.join(asha);
        room.join(ben);
        room.join(chen);
        asha.say("standup in 5");
        ben.whisper("chen", "coffee first?");
        chen.say("buy cheap SPAM now");

        System.out.println("\n=== 3. Air traffic control ===");
        ControlTower tower = new ControlTower();
        Flight ai101 = new Flight("AI101", tower), ba22 = new Flight("BA22", tower), lh7 = new Flight("LH7",
tower);
        ai101.requestLanding();
        ba22.requestLanding();
        lh7.requestLanding();
        ai101.landed();
        ba22.landed();
    }
}
```

Output:

```
=== 1. Shipping form: widgets coordinated by a mediator ===
  [submit] disabled
  [country] user picked Germany
  [methods] reloaded: [standard, express], default standard
  [price] standard shipping 5.00
  [submit] enabled
  [methods] user picked express
  [price] express shipping 15.00
  [mediator] order placed: Germany / express
  [country] user picked Antarctica
  [methods] reloaded: []
  [price] shipping not available
  [submit] disabled
  [submit] click ignored: button is disabled

=== 2. Chat room ===
  asha sees <room> asha joined
  asha sees <room> ben joined
  ben sees <room> ben joined
  asha sees <room> chen joined
  ben sees <room> chen joined
  chen sees <room> chen joined
  ben sees <asha> standup in 5
  chen sees <asha> standup in 5
  chen sees <ben (private)> coffee first?
  chen sees <room> message blocked by moderation

=== 3. Air traffic control ===
  [tower] AI101: cleared to land
  [tower] BA22: hold, runway busy with AI101
  [tower] LH7: hold, runway busy with AI101
  [tower] AI101 vacated the runway
  [tower] BA22: cleared to land
  [tower] BA22 vacated the runway
  [tower] LH7: cleared to land
```

One user action ("picked Germany") updated three components — and **no component called another**.

Mediator or Observer?

Both decouple objects. The difference is **what happens to the news**.

	Mediator	Observer
What travels	An event **and** the decision on what should follow	Only the event
Where reaction logic sits	In one place: the mediator	Spread across the observers, each deciding alone
Do the parties know each other?	No — not even how many peers exist	No; the subject only knows it has a list of subscribers
Flow	Both ways: colleagues report in, the mediator calls out to them	One way: the subject publishes, observers respond
Order of reactions	Written explicitly in the mediator — a reason to choose it	Not part of the contract; observers should not depend on each other's order

Practical test: does something need to be coordinated? "The country changed; anyone interested, update yourself" is Observer, and it is the lighter choice. "The country changed: refresh the methods, *after that* price the new default method, *after that* decide about submit" is sequenced coordination — Mediator territory. Short version: **with Observer the listeners decide; with Mediator the hub decides and gives orders.** (A mediator is often *implemented* using observer-style notifications from the components.)

Trade-offs

Pros	Cons
Every widget depends on exactly one thing, so links grow linearly instead of quadratically.	Every new rule lands in the mediator, so it keeps growing.
All interaction rules sit in one class you can read and unit-test.	Anyone changing the screen changes that same file, so merge conflicts pile up there.
Widgets are reusable because they no longer pull their neighbours with them.	To learn why a widget changed you read the mediator, not the widget.
A new widget costs one class plus one rule.	It can quietly become a **god object** (a class that knows and does too much).

Keeping the mediator from becoming a god object:

- **Split by area** — one mediator per screen section or workflow, not one per application. Two 80-line mediators are much easier to live with than one 160-line one.
- **Keep it to coordination.** Tax and pricing calculations belong in their own classes; the mediator decides *what to call*, not *how to compute* (a Facade has to respect the same boundary).
- **Reject shortcuts in code review.** A single widget-to-widget reference breaks the whole guarantee, and the compiler will not catch it.
- **Dispatch by table, not by `if`-chain.** In languages with first-class functions the mediator can keep a map from event (or sender) to handler function — e.g. a Python `dict` of `{event: method}` — so each rule is easy to find.

When not to use it

- **Only two objects interact.** Let them call each other; a coordinator for a single relationship adds a layer that does nothing.
- **The interactions are few and stable.** The pattern swaps many small, scattered links for one bigger central class; that only pays off once the tangle is large.
- **Nothing needs coordinating** — components react independently. Use Observer.
- **The mediator is already a 400-line god object.** More rules will not help; break it into smaller mediators first.

Common mistakes

1. **Components calling each other "just this once".**
2. **Computation inside the mediator** (tax, pricing), turning it into a god object.
3. **Confusing it with Facade.** A Facade is placed *in front of* a subsystem, and the subsystem's classes are unaware of it; calls flow one way. A Mediator is placed *among* peers that all know it and report to it; calls flow both ways.
4. **String-typed events everywhere** (`"country"`, `"method"`) that silently break on a typo. Use the sender reference, an enum, or typed events.

Real-world examples

- **UI controllers / view-models** coordinating several widgets (MVC controllers, MVVM view-models, Android `ViewModel` wiring). GoF's own example is a dialog box coordinating its widgets.
- **Chat servers** (Slack, WhatsApp): clients send messages to the server, which routes them to rooms and people and applies moderation.
- **Air traffic control** — the classic analogy.
- **Workflow / saga orchestrators** (e.g. Temporal, Camunda, AWS Step Functions): a checkout orchestrator invokes payment, stock reservation and shipping in sequence and holds the recovery plan for each failure. Ride-hailing dispatch (matching riders and drivers through a central service rather than drivers talking to riders directly) has the same shape.
- **Redux store** at application scale (components dispatch actions instead of calling each other; reducers hold the rules); **MediatR** in .NET popularised the name on the server side, using it to dispatch each request to

its handler.
- **Message brokers / event buses** at system scale: a service hands messages to the broker rather than calling other services; the broker's routing configuration is the mediator logic — and it can become a god object too.
- `java.util.Timer` and `Executor` / `ExecutorService` are often listed (including by the course) as JDK mediators: they stand between submitted tasks and the threads that run them, so neither side knows the other. Treat this as a *loose* fit — they do not coordinate peer colleagues the way the GoF dialog example does, so in an interview lead with UI controllers or orchestrators.

In the interview

> **30-second answer:** "Mediator swaps peer-to-peer references for a single hub. Widgets just report events to it; the hub owns the rules and instructs particular widgets. Four widgets go from six links to four, and a fifth gives five instead of ten. The dependencies aren't gone — they're concentrated in one class that I can read and test. The risk is a god object, so I split mediators per screen area or workflow and push computation into its own classes."

Follow-ups:

- *Mediator vs Observer?* With Observer each listener decides for itself; with Mediator the hub decides and instructs. When the sequence of reactions matters, pick Mediator.
- *Mediator vs Facade?* Facade simplifies a subsystem for outside callers (one-way, subsystem unaware). Mediator coordinates peers that know it (two-way).
- *How does it relate to Command?* A mediator often creates and dispatches commands.
- *How do you test a mediator?* Give it fake components and assert which calls it made for each event.
- *What about in microservices?* Orchestration (a central saga orchestrator) is mediator-style; choreography (services react to each other's events) is observer-style.

Self-check

1. **Q:** How many links do 5 fully connected components need, and how many with a mediator? **A:** 10 vs 5.
2. **Q:** Does a mediator remove coupling? **A:** No, it moves it into one class.
3. **Q:** What is the main failure mode? **A:** The mediator grows into a god object.
4. **Q:** Which way does traffic flow in a Mediator vs an Observer? **A:** Mediator two-way; Observer one-way.
5. **Q:** What should *not* go into a mediator? **A:** Business calculations — only coordination.

TL;DR

Peers talk through **one coordinator** that holds the interaction rules. Links become linear and components reusable, but the coupling **moves** into the mediator — keep it small and split it before it becomes a god object.

10.5 Memento

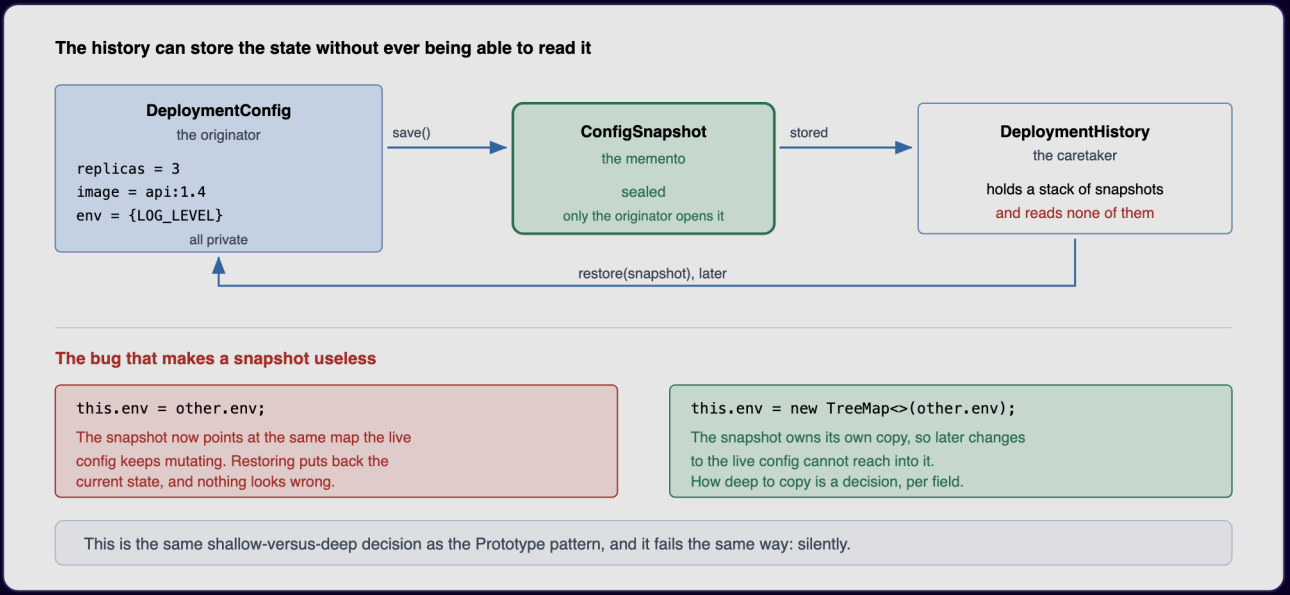


Figure – The originator (DeploymentConfig) saves a sealed snapshot; the caretaker (DeploymentHistory) stores it without being able to read it; below, the shared-map bug that makes a snapshot useless (course diagram)

What it is

Memento lets an object record its internal state in a **snapshot** and later **return** to that state – while nobody else gets to **see** what the snapshot contains.

GoF intent, in plain words: *without breaking encapsulation, capture and externalize an object's internal state so the object can be put back into that state later.* GoF participants: **Memento** (stores the state; offers a narrow interface to the caretaker and a wide one to the originator), **Originator** (creates a memento of its current state and uses one to restore itself) and **Caretaker** (keeps mementos safe but never operates on or examines their contents).

Everyday analogy: a **save game**. The game writes a save file; the menu lists the saves by name and date and lets you load one. The menu never needs to understand what is inside the file.

The problem it solves

You need a **rollback**: record the deployment configuration just before a risky change, and if the rollout misbehaves, bring the recorded version back.

The obvious way – something *outside* reads every field and stores them – breaks two things:

1. **Encapsulation.** The history class must now understand replicas, image names and environment variables. Every new config field forces a change in the history class, and details that were private are now exposed.
2. **Atomicity.** A restore done one field at a time can stop half-way. If three fields are written and an exception hits before the fourth, the config holds a combination that was never real.

With Memento the object **takes its own snapshot** and hands it out as a sealed token that **only it can open**. Restoring becomes **one call into the originator**, which is in full control of the restore and can make it all-or-nothing (our `restore` validates the token first, then assigns every field; nothing in between can fail half-way).

How it works

Three roles, with standard names interviewers use:

Role	Job	In our code
Originator	Owns the state; produces snapshots (`save`) and returns to them (`restore`). No other class can look inside a snapshot.	`DeploymentConfig`
Memento	Carries the state; opaque to everyone else.	`DeploymentConfig.Snapshot` (narrow) / private `State` record (wide)
Caretaker	Decides **when** to save and restore; never looks **inside**.	`DeploymentHistory`

Steps:

1. Caretaker asks the originator: `snapshot = config.save(label)`.
2. Caretaker stores the snapshot (usually on a stack).
3. Later, caretaker calls `config.restore(snapshot)`.
4. The originator copies the state back in one step.

How opacity is enforced in Java. GoF describes a **narrow interface** (what the caretaker sees) and a **wide interface** (what the originator sees). Our design does this with types: the public interface `Snapshot` exposes only `label()`; the real data is in a **private nested record** `State` that only `DeploymentConfig` can see. `restore` checks `instanceof State`, so even a forged `Snapshot` (see section 4 of the output) is refused. (Strictly, reflection or a debugger can still peek; Java access control protects against honest code, not a hostile JVM.) Other options by language: package-private accessors in Java; the originator declared a `friend` in C++; a private nested class in C#; Python and JavaScript offer no true enforcement, so it is a naming convention (leading underscore) plus code review – although a JavaScript closure *can* hide the contents in a way a class field cannot.

Structure (UML)

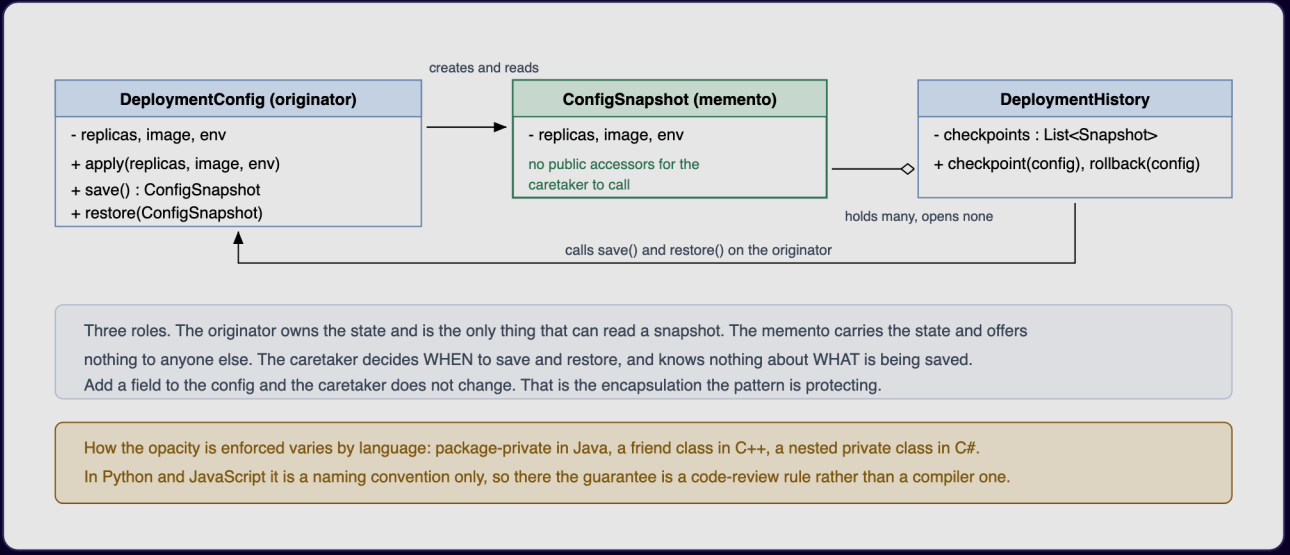
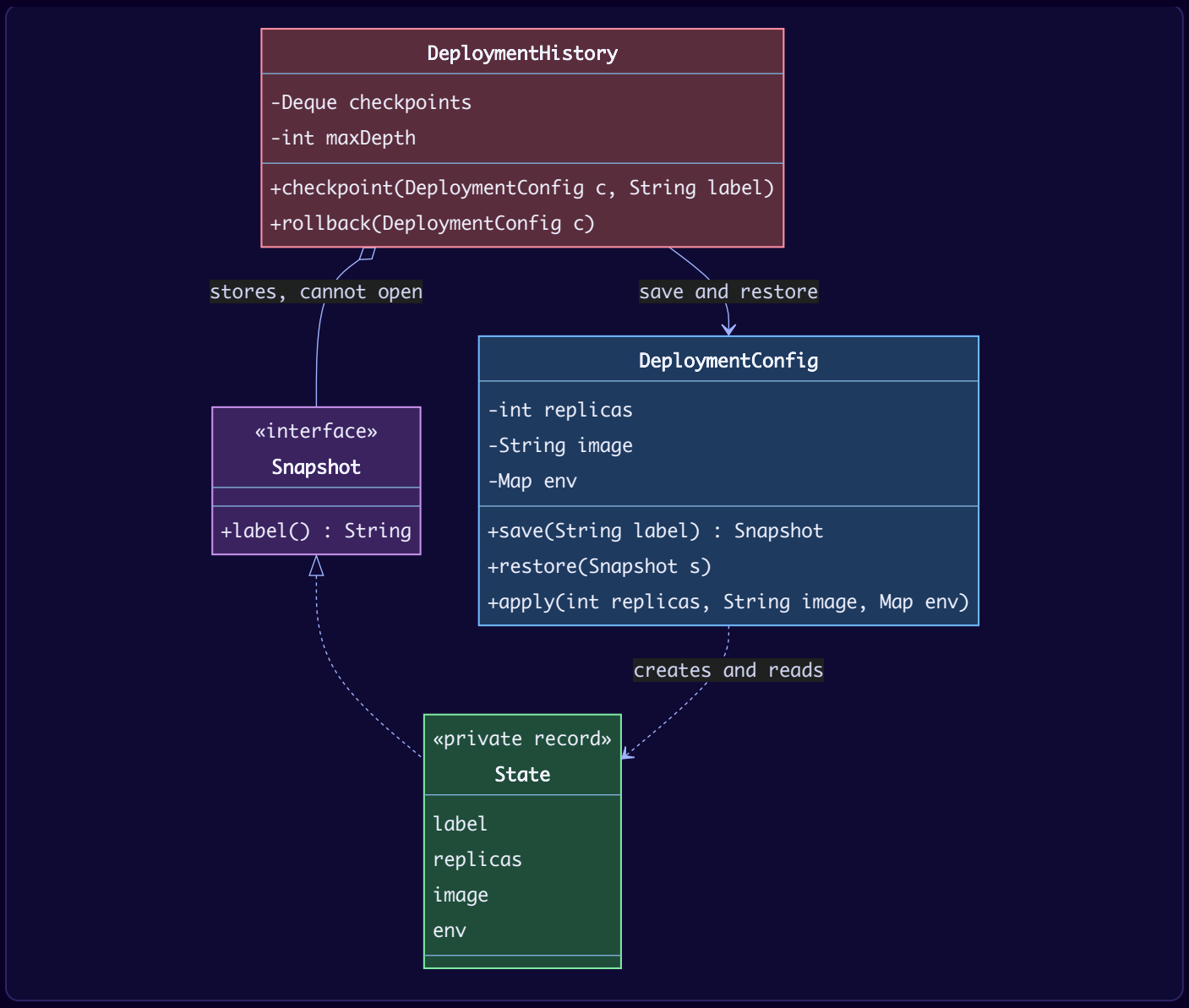
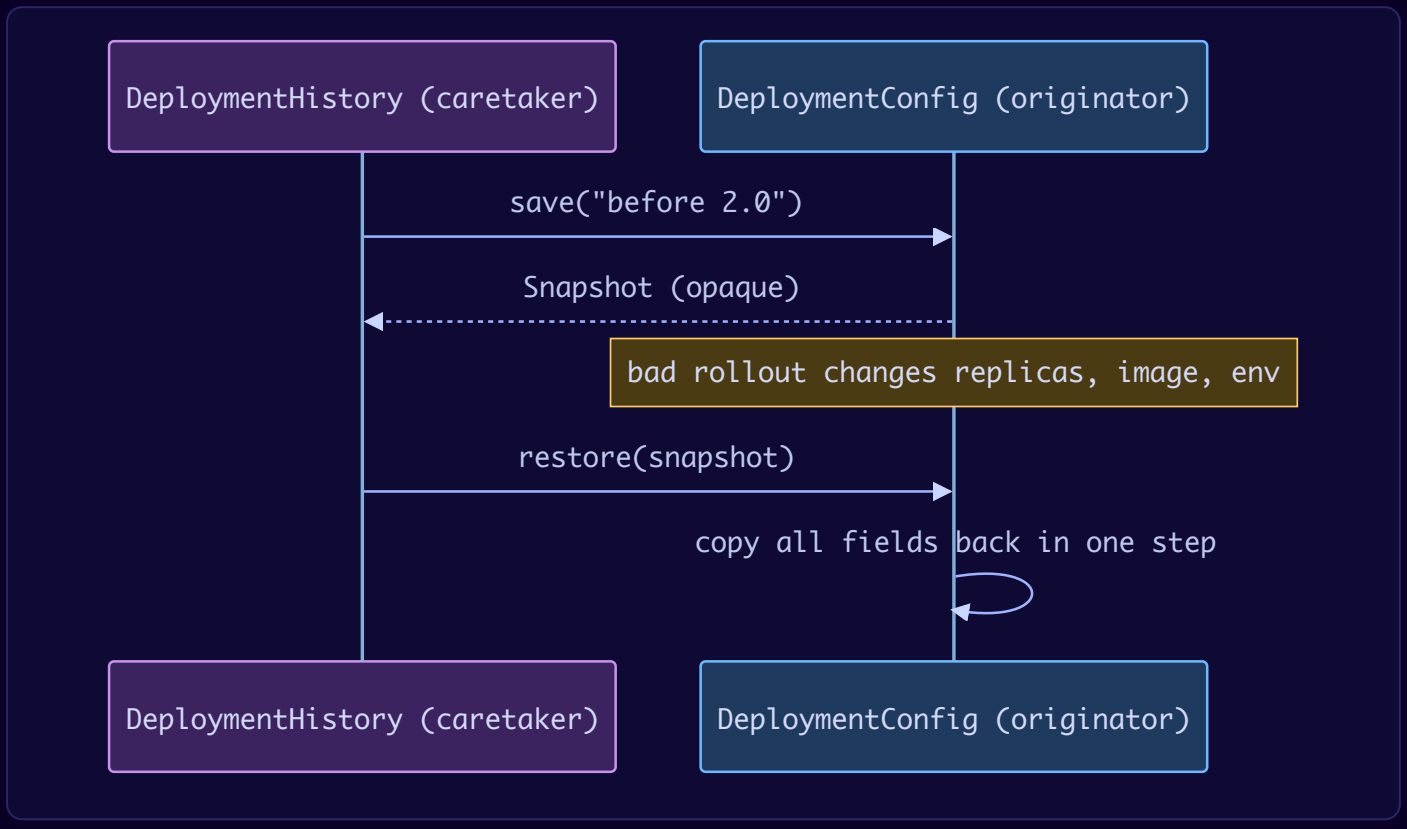


Figure – Class diagram: DeploymentConfig creates and restores ConfigSnapshot; DeploymentHistory holds a list of snapshots (course diagram)

In the diagram above, `DeploymentHistory` never refers to a single configuration field. That is the quick check for a real Memento: if the caretaker names the fields, you have only relocated them.

Our Java design:


DeploymentHistory
-Deque checkpoints
-int maxDepth
+checkpoint(DeploymentConfig c, String label)
+rollback(DeploymentConfig c)
stores, cannot open
save and restore
«interface»
Snapshot
+label() : String
DeploymentConfig
-int replicas
-String image
-Map env
+save(String label) : Snapshot
+restore(Snapshot s)
+apply(int replicas, String image, Map env)
creates and reads
«private record»
State
label
replicas
image
env


DeploymentHistory (caretaker)
DeploymentConfig (originator)
save("before 2.0")
Snapshot (opaque)
bad rollout changes replicas, image, env
restore(snapshot)
copy all fields back in one step
DeploymentHistory (caretaker)
DeploymentConfig (originator)

Java example

Points to notice in `lld.ch10.memento`:

- The `State` record's compact constructor does `env = Map.copyOf(env)`, so the snapshot **owns its own copy**. In section 1 the live map is changed *after* saving (`FEATURE_X`), and the rollback still restores the original.
- `DeploymentHistory` is **bounded** (`maxDepth`) and drops the oldest checkpoint when full.
- Section 3 shows the **shallow-copy bug** in isolation.

```java
// File: code/lld/ch10/memento/DeploymentConfig.java
package lld.ch10.memento;

import java.util.Map;
import java.util.TreeMap;

/** The ORIGINATOR: owns the state, and is the only class that can read a snapshot. */
public final class DeploymentConfig {

    /**
     * The NARROW interface the caretaker sees: a label, nothing else.
     * (GoF: caretaker gets a narrow interface, originator a wide one.)
     */
    public interface Snapshot {
        String label();
    }

    /** The WIDE interface: private, so only DeploymentConfig can open it. Immutable record. */
    private record State(String label, int replicas, String image, Map<String, String> env)
            implements Snapshot {
        State {
            env = Map.copyOf(env);   // DEEP enough: the snapshot owns its own copy, not the live map
        }
    }

    private int replicas;
    private String image;
    private final Map<String, String> env = new TreeMap<>();

    public DeploymentConfig(int replicas, String image, Map<String, String> env) {
        apply(replicas, image, env);
    }

    public void apply(int replicas, String image, Map<String, String> env) {
        this.replicas = replicas;
        this.image = image;
        this.env.clear();
        this.env.putAll(env);
    }

    public void setEnv(String key, String value) { env.put(key, value); }

    public Snapshot save(String label) {
        return new State(label, replicas, image, env);
    }

    /** One call restores everything: all-or-nothing. */
    public void restore(Snapshot snapshot) {
        if (!(snapshot instanceof State s)) {
            throw new IllegalArgumentException("not a snapshot produced by DeploymentConfig");
        }
        apply(s.replicas(), s.image(), s.env());
    }

    @Override
    public String toString() {
        return "replicas=" + replicas + " image=" + image + " env=" + env;
    }
}
```

```java
// File: code/lld/ch10/memento/DeploymentHistory.java
package lld.ch10.memento;

import java.util.ArrayDeque;
import java.util.Deque;

/** The CARETAKER: decides WHEN to save/restore, never WHAT is inside. Bounded to avoid leaks. */
public final class DeploymentHistory {
    private final Deque<DeploymentConfig.Snapshot> checkpoints = new ArrayDeque<>();
    private final int maxDepth;

    public DeploymentHistory(int maxDepth) { this.maxDepth = maxDepth; }

    public void checkpoint(DeploymentConfig config, String label) {
        if (checkpoints.size() == maxDepth) {
            DeploymentConfig.Snapshot dropped = checkpoints.removeLast();   // oldest
            System.out.println("  [history] full, dropped oldest checkpoint '" + dropped.label() + "'");
        }
        checkpoints.push(config.save(label));
        System.out.println("  [history] checkpoint '" + label + "' taken");
    }

    public void rollback(DeploymentConfig config) {
        if (checkpoints.isEmpty()) { System.out.println("  [history] nothing to roll back to"); return; }
        DeploymentConfig.Snapshot s = checkpoints.pop();
        config.restore(s);
        System.out.println("  [history] rolled back to '" + s.label() + "'");
    }
}
```

```java
// File: code/lld/ch10/memento/Main.java
package lld.ch10.memento;

import java.util.HashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Checkpoint, bad rollout, rollback ===");
        DeploymentConfig config = new DeploymentConfig(3, "api:1.4", Map.of("LOG_LEVEL", "info"));
        DeploymentHistory history = new DeploymentHistory(2);
        System.out.println("  live: " + config);

        history.checkpoint(config, "before 2.0");
        config.apply(10, "api:2.0", Map.of("LOG_LEVEL", "debug"));   // the risky rollout
        config.setEnv("FEATURE_X", "on");                             // mutates the live map after saving
        System.out.println("  live: " + config);
        history.rollback(config);
        System.out.println("  live: " + config + "   <- snapshot was a real copy");

        System.out.println("\n=== 2. Bounded history ===");
        history.checkpoint(config, "v1");
        history.checkpoint(config, "v2");
        history.checkpoint(config, "v3");      // drops v1
        history.rollback(config);
        history.rollback(config);
        history.rollback(config);

        System.out.println("\n=== 3. The shallow-copy bug, shown in isolation ===");
        Map<String, String> liveEnv = new HashMap<>(Map.of("LOG_LEVEL", "info"));
        Map<String, String> badSnapshot = liveEnv;                  // shares the live map
        Map<String, String> goodSnapshot = new HashMap<>(liveEnv);  // owns a copy
        liveEnv.put("LOG_LEVEL", "debug");
        System.out.println("  bad snapshot  = " + badSnapshot + "  <- changed with the live object");
        System.out.println("  good snapshot = " + goodSnapshot);

        System.out.println("\n=== 4. The caretaker cannot open a snapshot ===");
        DeploymentConfig.Snapshot s = config.save("peek");
        System.out.println("  caretaker can read only: label=" + s.label());
        try {
            config.restore(() -> "forged");                         // a Snapshot not made by the originator
        } catch (IllegalArgumentException e) {
            System.out.println("  forged snapshot refused: " + e.getMessage());
        }
    }
}
```

Output:

```
=== 1. Checkpoint, bad rollout, rollback ===
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}
  [history] checkpoint 'before 2.0' taken
  live: replicas=10 image=api:2.0 env={FEATURE_X=on, LOG_LEVEL=debug}
  [history] rolled back to 'before 2.0'
  live: replicas=3 image=api:1.4 env={LOG_LEVEL=info}   <- snapshot was a real copy

=== 2. Bounded history ===
  [history] checkpoint 'v1' taken
  [history] checkpoint 'v2' taken
  [history] full, dropped oldest checkpoint 'v1'
  [history] checkpoint 'v3' taken
  [history] rolled back to 'v3'
  [history] rolled back to 'v2'
  [history] nothing to roll back to

=== 3. The shallow-copy bug, shown in isolation ===
  bad snapshot  = {LOG_LEVEL=debug}  <- changed with the live object
  good snapshot = {LOG_LEVEL=info}

=== 4. The caretaker cannot open a snapshot ===
  caretaker can read only: label=peek
  forged snapshot refused: not a snapshot produced by DeploymentConfig
```

The bug that makes a snapshot useless

If the memento keeps a **pointer to the very map (or list) the originator goes on mutating**, you have not taken a snapshot: the "saved" copy drifts along with the live object, and a restore just re-applies today's state. Nothing throws and nothing looks wrong — until somebody notices the rollback had no effect. Always copy mutable parts (`Map.copyOf`, `new TreeMap<>(...)`, deep copies of nested objects). It is the same shallow-vs-deep copy decision as the Prototype pattern (Chapter 8).

Two more things to bound: history depth — keep a cap and evict the oldest entry, because a short undo list is useful while an endless one is just a memory leak — and **state that cannot be captured** (live sockets, open files, data owned by another system). A snapshot that claims to cover those will fail, usually long after the mistake was made.

Memento or Command undo?

	Memento	Command undo
Saved per step	A full copy of the state at that moment	The operation plus the little bit of state its reversal needs
How undo happens	Re-install an earlier copy	Execute the opposite operation
Memory cost per step	Proportional to the state size	Proportional to one operation
Correctness risk	Low — the copy was a genuine state	Every reverse operation must be written correctly
Prefer it when	Reversal is tricky or the state is small	Reversal is simple or the state is big

A drawing app that snapshots the whole canvas on every brush stroke runs out of memory — use Command ("stroke added, here are its points"). A wizard where step 3 changes six interacting fields is the opposite — six correct inverses are hard, one snapshot is easy. **Real systems combine them:** take a snapshot occasionally, log commands in between; to go back, restore the nearest snapshot and replay commands forward. Databases and event-sourced systems (Chapter 11) do exactly this.

Trade-offs

Pros	Cons
Rollback is possible while the internals stay private.	Memory use rises with every stored snapshot, quickly for large state.
New fields in the originator need no change in the caretaker.	Mutable parts must be deep-copied, or snapshots share live data.
The originator controls the restore, so it can be all-or-nothing, and it returns a state that really existed.	Some languages cannot truly hide the snapshot, only by convention.
Checkpoints, wizards, save games, transactions: one mechanism.	Some state (sockets, external systems) cannot be captured at all.

When not to use it

- **Large state that changes often** – use Command undo or periodic snapshots + replay.
- **State you cannot capture** (sockets, file handles, remote systems).
- **The object is immutable.** An old instance already *is* a perfect snapshot; holding a reference to it is enough (which is why functional code rarely names this pattern).
- **Each operation has an easy exact inverse** – per-step Command undo costs less memory.

Common mistakes

1. Storing references to mutable internals (the shallow-copy bug).
2. Letting the caretaker read or edit the snapshot's fields (encapsulation lost).
3. Restoring field by field from outside (not atomic).
4. Unbounded history.
5. Snapshotting things that cannot really be restored (connections, locks).

Real-world examples

- **Database transactions**: `BEGIN` ... `ROLLBACK`. The DB keeps enough information (undo logs) to put things back; your code never sees it.
- **Version control**: a Git commit is a snapshot of the tree; `git checkout` restores it, and the history (the caretaker) does not need to understand file contents to do its job.
- **Kubernetes Deployments** keep old ReplicaSets as revision history (bounded by `revisionHistoryLimit`, default 10), and `kubectl rollout undo` restores a previous pod template – a bounded caretaker, just like ours.
- **VM / container snapshots** and infrastructure state files that make `rollback` possible.
- **Game saves and checkpoints**; **form wizards** with a "back" button; **editor undo** for small documents.
- In Java, `Serializable` or a JSON mapper is often *how* a memento is stored when it must outlive the process; a `record` gives an immutable memento in one line.
- **Other languages**: in Python, `copy.deepcopy(self.__dict__)` is a quick memento and `dataclasses.replace` covers immutable objects; in JavaScript, `structuredClone(state)` deep-copies plain data.

In the interview

> **30-second answer:** "With Memento an object packs its state into a token that only the object itself can unpack. The originator produces and consumes snapshots, the memento holds the data, and the caretaker chooses when to save or roll back without ever inspecting the contents. Restoring is a single call the originator controls, so it can be all-or-nothing, and a new field in the object never touches the history code. I'd make sure the snapshot deep-copies mutable parts, bound the history, and compare it with Command undo: Memento stores state, Command stores actions; real systems combine periodic snapshots with a command log."

Follow-ups:

- *How do you enforce that the caretaker can't read the memento in Java?* A narrow public interface (e.g. only a label) implemented by a private nested record; or package-private accessors.

- *Memento vs Prototype?* Each copies state. A prototype clone becomes a live object you keep working with. A memento does nothing on its own; its sole purpose is to be given back to the originator.
- *Memento vs a backup?* A backup targets disaster recovery and lives outside the process (disk, another region). A memento is usually an in-memory checkpoint for undo. Same idea, very different lifetime and cost.
- *How would you implement undo for a large document?* Command undo for each edit, plus periodic snapshots so undo far back doesn't replay thousands of commands.
- *How does it relate to serialization?* Serialization is one common *storage format* for a memento; the pattern itself is about *access* – only the originator may look inside.

Self-check

1. **Q:** Name the three roles. **A:** Originator, memento, caretaker.
2. **Q:** Who may read the memento's contents? **A:** Only the originator.
3. **Q:** Why copy the `env` map in the snapshot? **A:** Otherwise the snapshot shares the live map and changes with it.
4. **Q:** When is Command undo better than Memento? **A:** When state is large and each action has an easy inverse.
5. **Q:** What does Kubernetes' `revisionHistoryLimit` correspond to? **A:** A bounded caretaker history.

TL;DR

The object **snapshots itself**; the caretaker **stores but cannot read**; restore is **one call the originator controls**. **Deep-copy** mutable parts, **bound** the history, and choose Memento (store state) vs Command (store actions) by which is smaller.

10.6 Interpreter

(Not covered by a lesson in the course beyond its one-line entry in the behavioral-patterns table; this section is from the GoF book and standard practice.)

What it is

Interpreter represents the **grammar** of a small language as a set of classes – **one class per grammar rule** – and evaluates a sentence of that language by building a **tree** of those objects and asking the root to `interpret` itself.

Everyday analogy: a calculator. `2 + 3 * 4` is a sentence in a tiny language. The calculator understands that `*` binds tighter than `+`, builds the structure `2 + (3 * 4)` in its head, and works it out from the inside.

Key terms:

- **Grammar** – the rules saying which sentences are valid, e.g. "an expression is terms joined by `+` or `-`".
- **DSL (domain-specific language)** – a small language for one job: search filters, pricing rules, feature-flag conditions, cron schedules.
- **AST (abstract syntax tree)** – the tree form of a sentence, with operators as inner nodes and numbers/variables as leaves.
- **Terminal expression** – a leaf rule (a number, a variable).
- **Nonterminal expression** – a rule made of other expressions (addition, multiplication).
- **Context** – outside information the evaluation needs, e.g. the current values of variables.

The problem it solves

Some problems keep coming back as *sentences*: "price * qty – discount", "age > 18 AND country = IN", "every weekday at 9:00". Hard-coding each rule in Java means a redeploy for every new rule. Letting business users write rules in a tiny language, and interpreting them, turns rules into **data**. Interpreter is the simplest object-oriented way to evaluate such a language when the grammar is **small**.

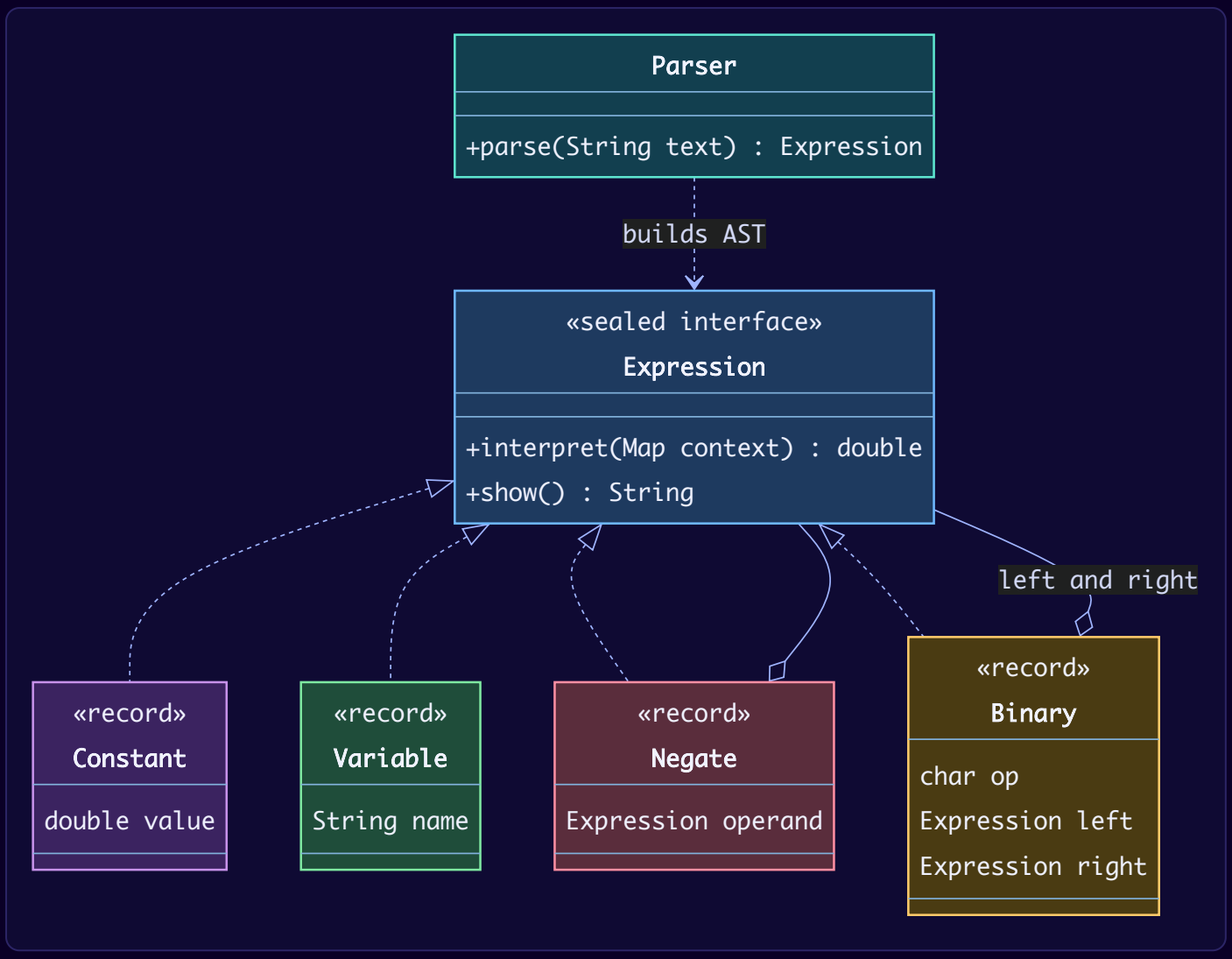
How it works

1. **Write the grammar** (here in EBNF, a standard notation):

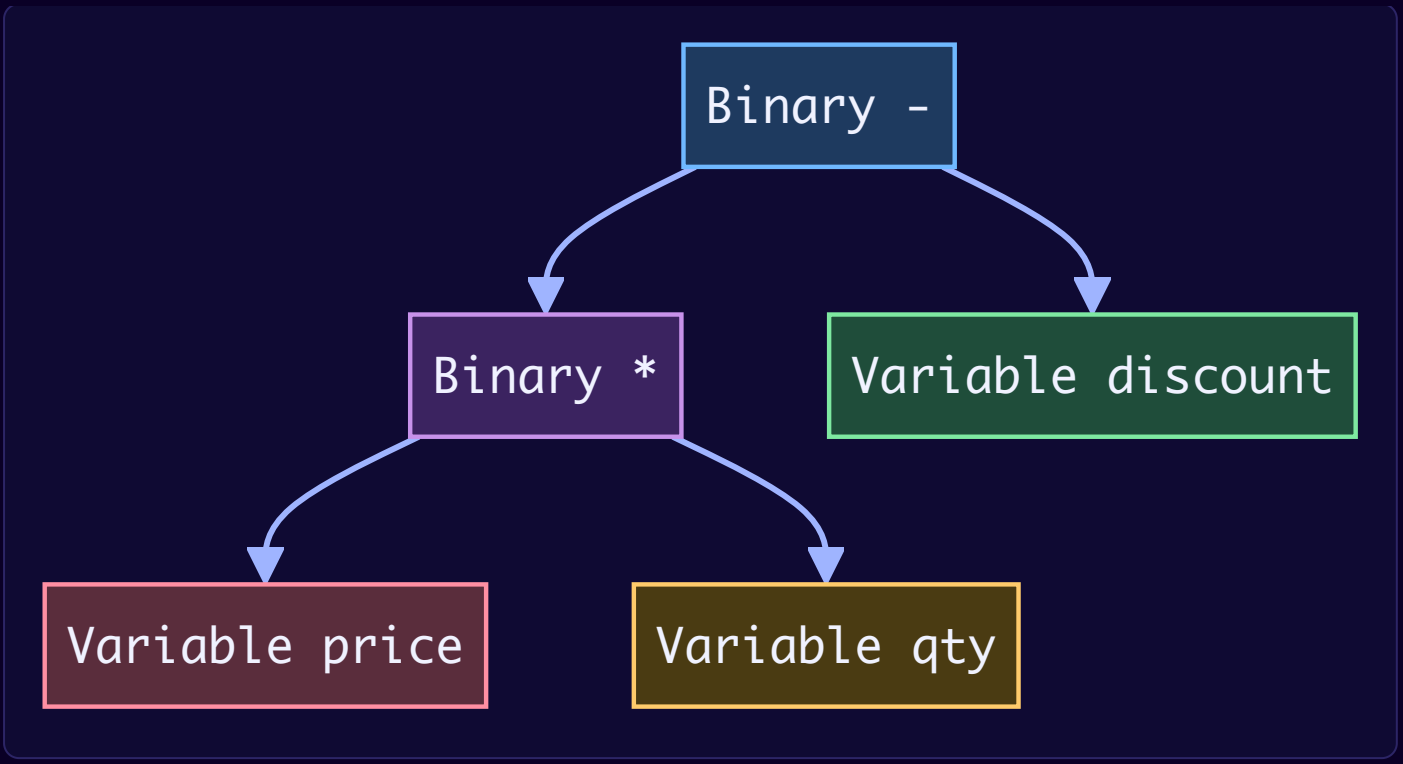
```
expr   := term   (('+' | '-') term)*
term   := factor (('*' | '/') factor)*
factor := NUMBER | NAME | '(' expr ')' | '-' factor
```

2. **One class per rule**: `Constant` and `Variable` (terminals), `Negate` and `Binary` (nonterminals), all implementing `Expression.interpret(context)`.
3. **Build the AST.** GoF deliberately leaves **parsing** out of the pattern; any parser will do. We use a small *recursive-descent* parser: one method per grammar rule, which calls the others.
4. **Interpret**: the root calls `interpret` on its children and combines their results (a *recursive* evaluation).
5. **Reuse the tree**: parse once, interpret many times with different contexts.

Structure (UML)



The AST for `price * qty - discount`:



Notice the structure is a **Composite** (Chapter 9): leaves and inner nodes share one interface. Interpreter is essentially "Composite + an `interpret` operation".

Java example

`lld.ch10.interpreter`: `Expression` is a **sealed interface** (only the four listed records may implement it), each record is one grammar rule, and `Parser` is a recursive-descent parser whose methods `expr`, `term` and `factor` mirror the grammar lines. Errors (unknown variable, division by zero, bad syntax) are reported with clear messages.

> **Why `double` here?** This is a general-purpose arithmetic calculator used to teach the pattern, and variable names like `price` are only sample data. A real pricing or promotion DSL must evaluate money with `BigDecimal` (and an explicit `RoundingMode` for division), exactly as the domain code elsewhere in this chapter does – `double` cannot represent values like 0.10 exactly.

```java
// File: code/lld/ch10/interpreter/Expression.java
package lld.ch10.interpreter;

import java.util.Map;

/**
 * AbstractExpression. One class per grammar rule; each knows how to interpret itself.
 * Grammar (EBNF):
 *   expr   := term   (('+' | '-') term)*
 *   term   := factor (('*' | '/') factor)*
 *   factor := NUMBER | NAME | '(' expr ')' | '-' factor
 */
public sealed interface Expression
        permits Expression.Constant, Expression.Variable, Expression.Negate, Expression.Binary {

    /** Evaluate against a context (variable name -> value). */
    double interpret(Map<String, Double> context);

    /** Print the tree back as fully-parenthesised text. */
    String show();

    /** TerminalExpression: a literal number. */
    record Constant(double value) implements Expression {
        public double interpret(Map<String, Double> ctx) { return value; }
        public String show() { return value == Math.rint(value) ? String.valueOf((long) value) :
String.valueOf(value); }
    }

    /** TerminalExpression: a variable looked up in the context. */
    record Variable(String name) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            Double v = ctx.get(name);
            if (v == null) throw new IllegalArgumentException("unknown variable '" + name + "'");
            return v;
        }
        public String show() { return name; }
    }

    /** NonterminalExpression: unary minus. */
    record Negate(Expression operand) implements Expression {
        public double interpret(Map<String, Double> ctx) { return -operand.interpret(ctx); }
        public String show() { return "(-" + operand.show() + ")"; }
    }

    /** NonterminalExpression: + - * / . Interprets its children, then combines. */
    record Binary(char op, Expression left, Expression right) implements Expression {
        public double interpret(Map<String, Double> ctx) {
            double l = left.interpret(ctx), r = right.interpret(ctx);
            return switch (op) {
                case '+' -> l + r;
                case '-' -> l - r;
                case '*' -> l * r;
                case '/' -> {
                    if (r == 0) throw new ArithmeticException("division by zero in " + show());
                    yield l / r;
                }
                default -> throw new IllegalStateException("unknown operator " + op);
            };
        }
        public String show() { return "(" + left.show() + " " + op + " " + right.show() + ")"; }
    }
}
```

```java
// File: code/lld/ch10/interpreter/Parser.java
package lld.ch10.interpreter;

/**
 * Recursive-descent parser: turns text into an Abstract Syntax Tree (AST).
 * Parsing is NOT part of the GoF Interpreter pattern; the pattern starts once the tree exists.
 * One method per grammar rule, mirroring the grammar in Expression.
 */
public final class Parser {
    private final String src;
    private int pos = 0;

    private Parser(String src) { this.src = src; }

    public static Expression parse(String text) {
        Parser p = new Parser(text);
        Expression e = p.expr();
        p.skipSpaces();
        if (p.pos != p.src.length()) throw p.error("unexpected '" + p.src.charAt(p.pos) + "'");
        return e;
    }

    private Expression expr() {
        Expression left = term();
        while (peek('+') || peek('-')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, term());   // left-associative
        }
        return left;
    }

    private Expression term() {
        Expression left = factor();
        while (peek('*') || peek('/')) {
            char op = src.charAt(pos++);
            left = new Expression.Binary(op, left, factor());
        }
        return left;
    }

    private Expression factor() {
        skipSpaces();
        if (pos >= src.length()) throw error("unexpected end of input");
        char c = src.charAt(pos);
        if (c == '-') { pos++; return new Expression.Negate(factor()); }
        if (c == '(') {
            pos++;
            Expression inner = expr();
            if (!peek(')')) throw error("missing ')'");
            pos++;
            return inner;
        }
        int start = pos;
        if (Character.isDigit(c) || c == '.') {
            while (pos < src.length() && (Character.isDigit(src.charAt(pos)) || src.charAt(pos) == '.'))
pos++;
            return new Expression.Constant(Double.parseDouble(src.substring(start, pos)));
        }
        if (Character.isLetter(c)) {
            while (pos < src.length() && Character.isLetterOrDigit(src.charAt(pos))) pos++;
            return new Expression.Variable(src.substring(start, pos));
        }
        throw error("unexpected '" + c + "'");
    }

    private boolean peek(char expected) {
```

```
        skipSpaces();
        return pos < src.length() && src.charAt(pos) == expected;
    }

    private void skipSpaces() {
        while (pos < src.length() && Character.isWhitespace(src.charAt(pos))) pos++;
    }

    private IllegalArgumentException error(String msg) {
        return new IllegalArgumentException(msg + " at position " + pos + " in \"" + src + "\"");
    }
}
```

```
// File: code/lld/ch10/interpreter/Main.java
package lld.ch10.interpreter;

import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        Map<String, Double> context = Map.of("price", 250.0, "qty", 3.0, "discount", 50.0, "a", 7.0, "b",
3.0);
        System.out.println("context: " + new java.util.TreeMap<>(context));

        List<String> inputs = List.of(
            "price * qty - discount",
            "2 + 3 * 4",
            "(2 + 3) * 4",
            "(a + b) * (a - b)",
            "-a + 10 / 4",
            "price / (qty - 3)",
            "price * tax",
            "2 + * 3");

        for (String text : inputs) {
            try {
                Expression ast = Parser.parse(text);          // build the tree once...
                double value = ast.interpret(context);        // ...interpret it (could be many times)
                System.out.printf("  %-24s AST %-32s = %s%n", text, ast.show(), value);
            } catch (IllegalArgumentException | ArithmeticException e) {
                System.out.printf("  %-24s ERROR: %s%n", text, e.getMessage());
            }
        }

        System.out.println("\nSame tree, different contexts (parse once, evaluate many):");
        Expression total = Parser.parse("price * qty - discount");
        for (double q : new double[] {1, 2, 4}) {
            System.out.println("  qty=" + (int) q + " -> " + total.interpret(Map.of("price", 250.0, "qty", q,
"discount", 50.0)));
        }
    }
}
```

Output:

```
context: {a=7.0, b=3.0, discount=50.0, price=250.0, qty=3.0}
  price * qty - discount   AST ((price * qty) - discount)       = 700.0
  2 + 3 * 4                AST (2 + (3 * 4))                    = 14.0
  (2 + 3) * 4              AST ((2 + 3) * 4)                    = 20.0
  (a + b) * (a - b)        AST ((a + b) * (a - b))              = 40.0
  -a + 10 / 4              AST ((-a) + (10 / 4))                = -4.5
  price / (qty - 3)        ERROR: division by zero in (price / (qty - 3))
  price * tax              ERROR: unknown variable 'tax'
  2 + * 3                  ERROR: unexpected '*' at position 4 in "2 + * 3"

Same tree, different contexts (parse once, evaluate many):
  qty=1 -> 200.0
  qty=2 -> 450.0
  qty=4 -> 950.0
```

Notice operator precedence (`2 + 3 * 4` = 14, but `(2 + 3) * 4` = 20) comes from the **grammar**: `term` handles `*` and `/` below `expr`, so they bind tighter.

Trade-offs

Pros	Cons
Easy to change and extend the grammar: a new rule is a new class.	One class per rule: large grammars become a jungle of classes.
Each rule's evaluation is small and testable.	Tree-walking is slow compared with compiling to bytecode.
Rules become data (stored in a DB, edited by users).	Adding a new *operation* (pretty-print, type-check, optimise) means touching every class – that is where **Visitor** (Chapter 11) comes in.
Parse once, evaluate many times.	You still need a parser, which the pattern does not give you.

When not to use it

- **The grammar is large or complex** (a real programming language, SQL). Use a parser generator such as **ANTLR** and a compiler/visitor design.
- **Performance matters** and expressions run millions of times – compile to bytecode or use an existing engine.
- **An existing language fits.** Regular expressions, SpEL, JSONPath, CEL, or a scripting engine may already do the job.
- **Security matters.** Never let users run a *general-purpose* script engine on your server; a small, restricted DSL is safer precisely because it can do only what its grammar allows.

Common mistakes

1. **Mixing parsing and evaluation** in one pass, so each evaluation re-parses the text.
2. **Ignoring precedence and associativity** (`8 - 3 - 2` must be `(8 - 3) - 2`; our `expr` loop builds left-associative trees).
3. **Poor error messages** – always report the position.
4. **Adding many operations to every node** instead of switching to Visitor or pattern matching.
5. **Growing a "tiny" DSL** until it is an accidental programming language.

Real-world examples

- `java.util.regex.Pattern`: compiling a regex builds an internal tree of node objects, and a `Matcher` walks it – a classic interpreter.
- **Spring Expression Language (SpEL)**: `SpelExpressionParser` parses text like `#user.age > 18` into an AST of `SpelNode`s, and `getValue()` evaluates it against a context. Jakarta Expression Language (EL) is similar.
- **SQL engines** evaluate `WHERE` clauses as expression trees; **spreadsheet formulas** (`=SUM(A1:A3)*2`) are a DSL.
- **Rule and pricing engines**: e-commerce promotions ("10% off if cart > 999 and category = electronics"), feature-flag targeting rules, and Google's **CEL** (Common Expression Language) used in Kubernetes validation rules and IAM conditions.

- Cron expressions (`0 9 * * MON-FRI`), e.g. Quartz's `CronExpression`.

In the interview

> **30-second answer:** "Interpreter models a small language's grammar as classes, one per rule — terminals like numbers and variables, nonterminals like addition. A sentence is parsed into an AST made of those objects and evaluated by calling `interpret` on the root with a context. It's really Composite plus an evaluate operation. It's great for small DSLs like pricing rules or filters; for big grammars I'd use a parser generator like ANTLR, and when I need many operations over the tree I'd switch to Visitor or sealed types with pattern matching."

Follow-ups:

- *Is parsing part of the pattern?* No. GoF assumes the tree already exists; parsing is a separate concern (hand-written recursive descent, or ANTLR).
- *Interpreter vs Visitor?* Interpreter puts evaluation **inside** the node classes. Visitor takes operations **out** into separate classes. A real interpreter usually starts as the first and becomes the second when a second pass (type-check, optimise, pretty-print) arrives.
- *Interpreter vs Composite?* Composite is the structure; Interpreter uses that structure to evaluate a language.
- *How would you make it faster?* Cache parsed trees; constant-fold (pre-compute parts with no variables); or compile to bytecode/lambdas once.
- *How do you design a rule engine for promotions?* A small grammar (comparisons, AND/OR/NOT, fields from the cart), rules stored as text, parsed once into trees and cached, evaluated per cart.

Self-check

1. **Q:** What is the difference between terminal and nonterminal expressions? **A:** Terminals are leaves (numbers, variables); nonterminals combine other expressions (operators).
2. **Q:** What is the "context"? **A:** External information needed for evaluation, such as variable values.
3. **Q:** Why does `2 + 3 * 4` give 14? **A:** The grammar puts `*` in `term`, below `expr`, so it binds tighter.
4. **Q:** Name a JDK class that is an interpreter. **A:** `java.util.regex.Pattern`.
5. **Q:** When should you not use it? **A:** Large grammars, performance-critical evaluation, or when an existing language fits.

TL;DR

One class per grammar rule; parse a sentence into an **AST**; evaluate by calling `interpret` recursively with a **context**. Perfect for **tiny DSLs**; for big grammars use a parser generator and Visitor.

10.7 Chapter recap: the six patterns side by side

Pattern	The object you create	The question it answers	Classic JDK / real example	Often confused with
Chain of Responsibility	A handler per check	*Who* should handle this request?	Servlet filters, logging handlers	Decorator (all layers run)
Command	An object per request	How do I store, queue or undo *this request*?	`Runnable`, `UndoManager`	Strategy (how vs what)
Iterator	A cursor object	How do I walk this without knowing its shape?	`Iterator`, `Spliterator`, `ResultSet`	Visitor (what to do at each node)
Mediator	One coordinator	How do many objects cooperate without knowing each other?	UI controllers, chat servers, saga orchestrators	Observer (broadcast only), Facade
Memento	A sealed snapshot	How do I go back to an earlier state safely?	DB rollback, Git, `kubectl rollout undo`	Command undo, Prototype
Interpreter	A node per grammar rule	How do I evaluate sentences of a small language?	`Pattern`, SpEL	Composite, Visitor

Interview combo facts to remember:

- The request flowing through a **Chain of Responsibility** is frequently a **Command** object.
- A **Mediator** often creates and dispatches **Commands**.
- **Memento + Command** together give scalable undo (snapshots + replay).
- **Iterator + Composite** walk trees; **Interpreter = Composite + evaluate**; when operations multiply, move to **Visitor** (Chapter 11).

Self-check (whole chapter)

1. **Q:** Which two GoF behavioral patterns are class-scoped (inheritance-based)? **A:** Template Method and Interpreter.
2. **Q:** A request must pass auth, then a rate limit, then validation, and any step can reject it. Which pattern? **A:** Chain of Responsibility.
3. **Q:** You need undo/redo for a text editor. Which patterns? **A:** Command (per edit), optionally Memento for periodic snapshots.
4. **Q:** Five form widgets depend on each other's values. Which pattern? **A:** Mediator.
5. **Q:** Your team needs business users to write discount rules. Which pattern? **A:** Interpreter (a small DSL), possibly with Specification (Chapter 11).

TL;DR

Behavioral patterns turn **hidden control flow** into **named objects**: handlers (CoR), requests (Command), cursors (Iterator), coordinators (Mediator), snapshots (Memento) and grammar nodes (Interpreter). Each one buys flexibility with indirection — use it when the flexibility is a real requirement, not "just in case".